Università degli Studi di Padova

Dipartimento di Matematica «Tullio Levi-Civita»

Corso di Laurea in Informatica



Interfaccia UI per dispositivi ESP32 per la visualizzazione di informazioni in tempo reale

Tesi di Laurea

Relatore
Prof. Zanella Marco

Laureando
Roberto Mariano Doroftei
Matricola 2111031

Anno Accademico 2025-2026

© Roberto Mariano Doroftei, Ottobre 2026. Tutti i diritti riservati. Tesi di Laurea: «Interfaccia UI per dispositivi ESP32 per la visualizzazione di informazioni in tempo reale», Università degli Studi di Padova, Dipartimento di Matematica «Tullio Levi-Civita», Corso di Laurea in Informatica.

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare, della durata di trecento ore, dal laureando Roberto Mariano Doroftei (matricola 2111031) presso l'azienda Spazio Dev S.r.l.. Lo stage è stato condotto sotto la supervisione del tutor aziendale Matteo Forzan, mentre il Prof. Zanella Marco ha ricoperto il ruolo di tutor accademico.

Questa tesi tratta la progettazione e lo sviluppo di un'interfaccia utente su microcontrollori ESP32 dotati di display, finalizzata alla visualizzazione in tempo reale delle informazioni gestite da RelAi, il CRM aziendale di Spazio Dev S.r.l.. Il sistema permette di esporre dati chiave quali codici QR, prenotazioni di eventi, il contatore degli iscritti a un'organizzazione e il contatore dei partecipanti a un evento direttamente presso le sedi delle organizzazioni clienti.

Lo scopo del progetto è duplice: da un lato, realizzare un firmware robusto e ottimizzato per dispositivi *embedded* con rigidi vincoli di memoria e risorse grafiche; dall'altro, implementare una connettività affidabile tramite provisioning Wi-Fi, gestione della cache locale dei dati e sincronizzazione periodica con le API aziendali, garantendo la continuità operativa anche in condizioni di assenza di rete. L'elaborato è articolato in sette capitoli, seguiti dal glossario e dai riferimenti bibliografici e sitografici. Il primo capitolo introduce il contesto aziendale, gli obiettivi del progetto e l'organizzazione dello stage. Il secondo descrive i processi e le metodologie adottate; il terzo presenta gli utenti, i casi d'uso e i requisiti. Il quarto illustra le tecnologie hardware e software, il quinto tratta la progettazione e la codifica del firmware e il sesto descrive le attività di verifica e validazione. Il settimo capitolo presenta il consuntivo, i risultati, i limiti e gli sviluppi futuri del progetto.

Ringraziamenti

Indice

Sommario iii
Ringraziamenti iv
Indice v
Elenco delle figure vii
Elenco delle tabelle viii
Elenco dei codici sorgenti ix
1 Introduzione ..... 1.
1.1 L'azienda ..... 1.
1.2 Il progetto ..... 1.
1.3 Obiettivi dello stage ..... 2.
1.4 Perimetro e vincoli ..... 2.
1.5 Organizzazione dello stage ..... 2.
1.6 Organizzazione del testo ..... 3.
2 Processi, metodologie e strumenti ..... 5.
2.1 Pianificazione e ripartizione delle attività ..... 5.
2.2 Requisiti e obiettivi ..... 7.
2.2.1 Requisiti Obbligatori (O) ..... 7.
2.2.2 Requisiti Desiderabili (D) ..... 7.
2.2.3 Requisiti Facoltativi (F) ..... 8.
2.3 Modello Agile e gestione iterativa ..... 8.
2.4 Issue Tracking e Controllo di Versione ..... 9.
2.5 Analisi preventiva dei rischi ..... 9.
3 Analisi dei requisiti ..... 13.
3.1 Analisi degli utenti ..... 13.
3.2 Casi d'uso ..... 13.
3.2.1 Elenco dei casi d'uso ..... 14.
3.2.2 Descrizione dettagliata dei casi d'uso ..... 14.
3.3 Requisiti del sistema e stato di completamento ..... 27.
3.3.1 Requisiti funzionali e stato di attuazione ..... 28.
3.3.2 Requisiti non funzionali, di qualità e vincolo ..... 29.
3.3.3 Conclusione sulla soddisfazione dei requisiti ..... 30.
4 Tecnologie utilizzate ..... 32.
4.1 Piattaforma hardware ..... 32.
4.2 Linguaggio C++ ..... 33.
4.3 PlatformIO ..... 34.
4.4 LVGL e interfaccia grafica ..... 35.
4.5 Connettività e protocolli ..... 35.
4.6 Persistenza e gestione dei dati ..... 36.
4.7 Controllo di versione e documentazione ..... 36.

4.7.1 Gitea ..... 37.
4.7.2 Typst per la redazione della tesi ..... 37.
4.8 Riepilogo dello stack tecnologico ..... 38.
5 Progettazione e codifica ..... 41.
5.1 Struttura del firmware ..... 41.
5.2 Backend: avvio, task e accesso condiviso ..... 42.
5.3 Backend: provisioning Wi-Fi e server locale ..... 43.
5.4 Backend: sincronizzazione con RelAi ..... 45.
5.4.1 Heartbeat e telemetria ..... 45.
5.4.2 Cache degli eventi e persistenza ..... 46.
5.4.3 WebSocket e comandi remoti ..... 47.
5.4.4 Logo dell'organizzazione ..... 48.
5.5 Frontend sul display: LVGL ..... 48.
5.5.1 Struttura e navigazione delle schermate ..... 50.
5.6 Frontend web: configurazione e diagnostica ..... 51.
5.7 Aspetti di sicurezza e limiti osservati ..... 51.
6 Verifica e validazione ..... 54.
6.1 Ambiente e modalità di prova ..... 54.
6.2 Avvio e configurazione della rete ..... 54.
6.3 Sincronizzazione con RelAi e funzionamento offline ..... 55.
6.4 Verifica dell'interfaccia sui dispositivi fisici ..... 56.
6.5 Anomalie emerse e interventi correttivi ..... 57.
6.6 Esito della validazione e limiti residui ..... 57.
7 Conclusioni ..... 59.
7.1 Consuntivo del progetto ..... 59.
7.2 Raggiungimento degli obiettivi ..... 59.
7.3 Competenze acquisite ..... 60.
7.4 Limiti e sviluppi futuri ..... 61.
7.5 Valutazione conclusiva ..... 61.
7.6 Consuntivo finale ..... 62.
7.7 Raggiungimento degli obiettivi ..... 62.
7.8 Conoscenze acquisite ..... 62.
7.9 Valutazione personale ..... 62.
7.10 Valutazione personale ..... 62.
8 Glossario ..... 63.
Riferimenti bibliografici e sitografici ..... 72.
Documentazione tecnica e piattaforme ..... 72.
Librerie e driver hardware ..... 72.
Strumenti di supporto ..... 73.

Elenco delle figure

Figura 1.1 Logo di Spazio Dev S.r.l. ..... 1.
Figura 3.1 Diagramma del caso d'uso UC1: Configurazione della connessione Wi-Fi ..... 14.
Figura 3.2 Diagramma del caso d'uso UC1.1: Nessuna rete disponibile . 15.
Figura 3.3 Diagramma del caso d'uso UC1.2: Credenziali non valide . . . 16.
Figura 3.4 Diagramma del caso d'uso UC2: Consultazione diagnostica locale del dispositivo ..... 17.
Figura 3.5 Diagramma del caso d'uso UC3: Consultazione stato e telemetria remota tramite CRM RelAi ..... 18.
Figura 3.6 Diagramma del caso d'uso UC4: Condivisione Wi-Fi tramite QR Code ..... 19.
Figura 3.7 Diagramma del caso d'uso UC5: Riavvio locale del dispositivo ..... 20.
Figura 3.8 Diagramma del caso d'uso UC6: Riavvio remoto del dispositivo ..... 21.
Figura 3.9 Diagramma del caso d'uso UC7: Reset di sistema locale ..... 22.
Figura 3.10 Diagramma del caso d'uso UC8: Reset di sistema remoto . . . 23.
Figura 3.11 Diagramma del caso d'uso UC9: Aggiornamento remoto dei dati tramite CRM RelAi ..... 24.
Figura 3.12 Diagramma del caso d'uso UC9.1: Gestione del fallback in assenza di connettività ..... 25.
Figura 3.13 Diagramma del caso d'uso UC10: Navigazione tra le schermate informative ..... 26.
Figura 3.14 Diagramma del caso d'uso UC11: Visualizzazione dei dati nei pannelli informativi a display ..... 27.
Figura 4.1 Scheda ESP32-S3 con display rotondo da 1,75 pollici utilizzata come dispositivo principale ..... 33.
Figura 4.2 Scheda ESP32-S3 con display quadrato da 1,8 pollici utilizzata per verificare il requisito facoltativo multi-display ..... 33.
Figura 4.3 Logo del linguaggio di programmazione C++ ..... 34.
Figura 4.4 Logo dell'ambiente di sviluppo PlatformIO ..... 34.
Figura 4.5 Esempio dell'interfaccia realizzata con LVGL sul display del dispositivo ..... 35.
Figura 4.6 Logo della piattaforma Gitea ..... 37.
Figura 4.7 Logo del sistema di composizione Typst ..... 37.

Elenco delle tabelle

Tabella 3.1 Requisiti funzionali e stato di completamento ..... 28.
Tabella 3.2 Requisiti non funzionali e di vincolo ..... 29.
Tabella 4.1 Riepilogo delle tecnologie utilizzate ..... 38.

Elenco dei codici sorgenti

Codice 5.1 Funzione `setup()` dell'ESP32-S3 ..... 42.
Codice 5.2 Ciclo principale del task di rendering ..... 43.
Codice 5.3 Avvio dell'Access Point e del DNS captive (versione sintetizzata) ..... 44.
Codice 5.4 Registrazione degli endpoint del server HTTP locale ..... 44.
Codice 5.5 Invio dell'heartbeat con telemetria (versione sintetizzata) .... 45.
Codice 5.6 Aggiornamento e salvataggio persistente degli eventi (versione sintetizzata) ..... 46.
Codice 5.7 Inizializzazione di display, touch e LVGL (versione sintetizzata) ..... 49.
Codice 5.8 Lettura del touch con debounce nella variante con controller CST820 ..... 49.

1 Introduzione

La gestione operativa delle organizzazioni e il monitoraggio in tempo reale delle attività aziendali richiedono spesso di coordinare flussi informativi complessi, tradizionalmente confinati all'interno di piattaforme software gestionali o pannelli web di controllo. Quando sorge l'esigenza di rendere questi indicatori immediatamente accessibili in contesti fisici distribuiti — come uffici, postazioni di accoglienza o spazi espositivi — l'utilizzo di soluzioni hardware dedicate consente di superare l'attrito dei sistemi tradizionali, offrendo informazioni visive immediate senza la necessità di postazioni informatiche complesse.

Questo elaborato presenta il progetto di stage svolto presso l'azienda **Spazio Dev S.r.l.**, che prevede la realizzazione e lo sviluppo di un prototipo di dispositivo interattivo basato su microcontrollore e display, progettato per integrarsi direttamente con la piattaforma CRM **RelAi**. Il progetto prende come riferimento i requisiti operativi e l'architettura aziendale, ponendosi l'obiettivo di verificare la fattibilità e l'efficacia di un'interfaccia *embedded* per la visualizzazione in tempo reale di dati chiave quali iscrizioni, metriche di eventi e codici di servizio attraverso un prototipo funzionante.

1.1 L'azienda

Spazio Dev S.r.l. (Figura 1.1) opera nel settore dello sviluppo software e della consulenza informatica, offrendo soluzioni tecnologiche avanzate per la gestione aziendale e dei flussi operativi. Il progetto di stage si inserisce nel contesto dell'ecosistema software aziendale, integrandosi con la piattaforma CRM RelAi per estendere le funzionalità di monitoraggio e interazione visiva direttamente su dispositivi *embedded* installati presso gli utenti finali

Figura 1.1: Logo di Spazio Dev S.r.l.

1.2 Il progetto

L'attività di stage riguarda la progettazione e lo sviluppo di una piattaforma firmware per microcontrollori ESP32 dotati di display. I dispositivi realizzati devono essere in grado di comunicare con il CRM RelAi e di presentare informazioni aggiornate in tempo reale mediante un'interfaccia grafica semplice e

facilmente leggibile.
La soluzione proposta prevede l'utilizzo di pannelli informativi intelligenti capaci di visualizzare differenti tipologie di contenuto, tra cui codici QR, informazioni relative agli eventi e dati statistici riguardanti organizzazioni e partecipanti. Il progetto comprende inoltre tutti gli aspetti necessari alla configurazione, all'integrazione e alla gestione operativa dei dispositivi.

1.3 Obiettivi dello stage

Lo stage si pone l'obiettivo di sviluppare una soluzione affidabile e facilmente utilizzabile, in grado di collegare il mondo dei sistemi gestionali aziendali con quello dei dispositivi embedded.
Le attività previste comprendono:

- l'analisi dei requisiti funzionali e tecnici del sistema;
- lo studio della piattaforma hardware ESP32 e delle tecnologie software necessarie;
- la progettazione dell'architettura del sistema;
- lo sviluppo dell'interfaccia grafica e dei meccanismi di comunicazione con il CRM;
- la verifica del corretto funzionamento della soluzione mediante attività di test;
- la produzione della documentazione tecnica relativa al progetto.

Nel corso delle prime settimane di attività è stata svolta un'intensa fase di analisi e studio preliminare, finalizzata alla comprensione della piattaforma hardware, delle librerie software e delle modalità di integrazione con il sistema RelAi. Tale fase costituisce la base per le successive attività di sviluppo e validazione previste dal piano di lavoro.

1.4 Perimetro e vincoli

Lo sviluppo su microcontrollore ESP32 impone vincoli rigorosi in termini di risorse computazionali e gestione della memoria. Il sistema deve inoltre garantire un livello elevato di robustezza operativa in esercizio, gestendo scenari di disconnessione della rete Wi-Fi o interruzioni temporanee della comunicazione con le API di RelAi attraverso meccanismi di cache locale, riconnessione automatica e controllo tramite *watchdog*.

1.5 Organizzazione dello stage

Il piano di attività ha una durata complessiva di 300 ore, suddivise indicati-

vamente in otto settimane (dal 21 settembre al 20 novembre 2026). Le prime settimane sono dedicate all'analisi preliminare, allo studio della piattaforma hardware ESP32 e delle librerie grafiche, per poi procedere con lo sviluppo incrementale del provisioning di rete, del client API, del rendering dei pannelli e delle funzioni di diagnostica e test finali.

1.6 Organizzazione del testo

- Il **secondo capitolo** descrive i processi e le metodologie di sviluppo adottati.
- Il **terzo capitolo** presenta l'analisi degli utenti, i casi d'uso e i requisiti del sistema.
- Il **quarto capitolo** descrive le tecnologie hardware e software utilizzate per lo sviluppo del prototipo.
- Il **quinto capitolo** tratta la progettazione architetturale e la codifica del firmware ESP32.
- La sezione finale raccoglie i riferimenti bibliografici e sitografici.

Gli acronimi sono sciolti alla prima occorrenza e i termini tecnici o stranieri sono evidenziati in *corsivo*.

4.

2 Processi, metodologie e strumenti

Il presente capitolo approfondisce il quadro metodologico e gli strumenti adottati nel corso delle 300 ore di stage svolto presso **Spazio Dev S.r.l.** (con sede a Tombolo, Padova). L'azienda è una realtà fortemente orientata allo sviluppo software di qualità e all'innovazione tecnologica, che promuove un ambiente collaborativo basato sulla condivisione delle competenze e sul miglioramento continuo.
Per affrontare la complessità dello sviluppo *embedded* e garantire standard qualitativi coerenti con le esigenze aziendali, l'attività è stata pianificata seguendo un approccio strutturato. L'integrazione del prototipo basato su microcontrollore ESP32 con la piattaforma CRM RelAi ha richiesto non solo competenze tecniche di programmazione e interfacciamento con API, ma anche una rigorosa aderenza ai flussi di lavoro interni, garantendo il rispetto delle tempistiche e un costante allineamento sugli obiettivi strategici del prodotto.

2.1 Pianificazione e ripartizione delle attività

Il percorso di stage è stato distribuito nell'arco di otto settimane (sette settimane da 40 ore e una settimana finale da 20 ore), per un totale di 300 ore complessive. Le attività sono state pianificate bilanciando la fase di studio iniziale, lo sviluppo del firmware e la validazione finale. La progressione temporale si articola nelle seguenti fasi:

- Prima Settimana - Analisi iniziale e setup (40 ore): onboarding sul progetto, configurazione dell'ambiente di lavoro e del repository, raccolta dei requisiti con il tutor e comprensione dei casi d'uso principali.
- Seconda Settimana - Studio dell'ESP32, del display e delle librerie grafiche (40 ore): studio dell'architettura del microcontrollore e del display, configurazione della toolchain e valutazione delle librerie grafiche e per i codici QR.
- Terza Settimana - Configurazione iniziale del dispositivo: provisioning WiFi e reset (40 ore): implementazione dell'avvio in modalità access point con portale di configurazione, salvataggio persistente e reset alle impostazioni di fabbrica.
- Quarta Settimana - Abbinamento a RelAi, client API e cache locale (40 ore): sviluppo della procedura di abbinamento al CRM, del client verso le API, della gestione degli errori e della cache locale dell'ultimo dato valido.
- Quinta Settimana - Pannello codice QR e gestione delle schermate (40 ore): generazione e rendering del codice QR e realizzazione della macchina a stati per la gestione delle schermate di stato e di errore.

- Sesta Settimana - Pannelli contatori e prenotazioni, rotazione automatica (40 ore): sviluppo dei pannelli relativi a iscritti, partecipanti e prenotazioni, con rotazione automatica e rifinitura grafica.
- Settima Settimana - Configurazione remota, diagnostica e robustezza (40 ore): recupero della configurazione da RelAi, esposizione della pagina di diagnostica locale e introduzione di meccanismi di robustezza come il *watchdog* e la riconnessione automatica.
- Ottava Settimana - Test, rifinitura e confronto finale (20 ore): esecuzione dei test funzionali e di durata prolungata, correzione dei bug, stesura della relazione finale e preparazione della demo conclusiva.

A supporto della pianificazione temporale, la ripartizione analitica delle ore in base all'impegno richiesto dai singoli moduli è definita come segue:

- **18 ore:** Onboarding, analisi dei requisiti e definizione dei pannelli informativi con il tutor aziendale.
- **14 ore:** Setup dell'ambiente di sviluppo (toolchain ESP32, repository, strumenti di flashing e di test).
- **30 ore:** Studio della piattaforma ESP32, del display e delle librerie grafiche e per la generazione dei codici QR.
- **22 ore:** Progettazione dei layout dei pannelli e analisi delle API di RelAi e dei dati da visualizzare.
- **38 ore:** Provisioning della rete Wi-Fi con portale di configurazione, salvataggio persistente delle impostazioni e reset del dispositivo.
- **28 ore:** Abbinamento del dispositivo a RelAi, gestione delle credenziali di accesso e del ciclo di vita dell'associazione.
- **28 ore:** Client verso le API, aggiornamento periodico dei dati, cache locale e sincronizzazione dell'ora.
- **45 ore:** Rendering dei pannelli sul display (codice QR, contatori e prenotazioni evento) e gestione delle schermate di stato.
- **18 ore:** Configurazione remota dei pannelli da RelAi e pagina di diagnostica locale del dispositivo.
- **14 ore:** Robustezza del dispositivo (watchdog, riconnessione, riavvio automatico) e predisposizione dell'aggiornamento del firmware.
- **25 ore:** Test funzionali e di durata sul dispositivo, bug fixing e miglioramenti grafici.
- **20 ore:** Documentazione tecnica, relazione finale e preparazione della presentazione.

2.2 Requisiti e obiettivi

Per dimensionare correttamente il progetto rispetto alle trecento ore di stage e monitorare il grado di completamento delle funzionalità, gli obiettivi e i requisiti sono stati classificati in tre livelli gerarchici:

2.2.1 Requisiti Obbligatori (O)

- **O01 (Configurazione e Provisioning):** Realizzare la procedura di configurazione iniziale del dispositivo, comprendendo il provisioning della rete Wi-Fi tramite portale di configurazione, il salvataggio persistente delle impostazioni e la funzione di reset alle impostazioni di fabbrica.
- **O02 (Integrazione CRM e API):** Sviluppare l'abbinamento del dispositivo al CRM RelAi e implementare il client verso le API aziendali, garantendo l'aggiornamento periodico dei dati e la cache locale dell'ultimo valore valido.
- **O03 (Visualizzazione e Gestione Stati):** Visualizzare sul display il codice QR, il contatore degli iscritti, il contatore dei partecipanti e le prenotazioni dell'evento, implementando schermate dedicate per gestire gli stati di errore e l'assenza di rete.
- **O04 (Robustezza e Affidabilità):** Garantire la stabilità operativa e l'affidabilità del dispositivo in condizioni d'uso prolungate, sottoponendo il sistema a rigorosi test funzionali e di stress prima della conclusione dello stage.

2.2.2 Requisiti Desiderabili (D)

- **D01 (Configurazione Remota):** Permettere la configurazione remota da RelAi dei pannelli mostrati e dell'intervallo di aggiornamento, evitando la necessità di riflashare il dispositivo.
- **D02 (Diagnostica Locale):** Esporre una pagina di diagnostica locale servita direttamente dal dispositivo, utile per monitorare lo stato della rete, l'ultima sincronizzazione, la versione del firmware e per attivare azioni di riavvio e reset.
- **D03 (Rotazione Automatica):** Introdurre la rotazione automatica tra più pannelli attivi configurando intervalli di durata personalizzabili.
- **D04 (Aggiornamento Firmware OTA):** Predisporre meccanismi di aggiornamento del firmware over-the-air (OTA) con controllo della versione disponibile.

2.2.3 Requisiti Facoltativi (F)

- **F01 (Aggiornamento Near-Real-Time):** Sostituire o integrare l'aggiornamento periodico con un flusso di dati *near-real-time* basato su protocolli come WebSocket o MQTT.
- **F02 (Check-in Partecipanti):** Gestire la registrazione del check-in dei partecipanti a un evento mostrando sul pannello un codice QR identificativo, che il socio può inquadrare con il proprio smartphone per completare la registrazione lato server su RelAi.
- **F03 (Supporto Multi-display e Temi):** Estendere il supporto ad altri modelli di display e introdurre la personalizzazione grafica dei pannelli tramite temi e loghi dell'organizzazione.

2.3 Modello Agile e gestione iterativa

L'organizzazione del lavoro ha ripreso i principi chiave delle metodologie **Agile**, suddividendo lo sviluppo complessivo in cicli iterativi brevi orientati al rilascio incrementale delle funzionalità. Questo approccio ha permesso di validare progressivamente i singoli componenti del sistema — dalla connettività Wi-Fi al rendering grafico su display — riducendo i rischi di regressione e consentendo correzioni tempestive in corso d'opera.
La comunicazione e la collaborazione con il tutor aziendale (Matteo Forzan) sono state strutturate attraverso incontri periodici fissati con cadenza regolare (due o tre volte a settimana), articolati secondo una precisa suddivisione dei ruoli operativi:

- **Primo incontro (inizio settimana):** dedicato alla pianificazione e revisione dei task settimanali, con l'individuazione dei pannelli o delle funzionalità da realizzare, la stima dell'impegno richiesto e l'assegnazione delle priorità operative in base agli obiettivi dello stage.
- **Incontro intermedio (metà settimana):** volto all'analisi tecnica e alla risoluzione di dubbi o criticità emerse durante lo sviluppo, con particolare focus sui colli di bottiglia legati alla connettività del dispositivo e al rendering sul display.
- **Ultimo incontro (fine settimana):** finalizzato alla verifica delle mansioni svolte attraverso test empirici e prove dirette dei pannelli sul dispositivo fisico, alla raccolta di feedback costruttivi e alla validazione dello stato di avanzamento rispetto al cronoprogramma.

In aggiunta ai momenti di confronto, il codice sorgente del firmware ESP32 è stato sottoposto a regolari sessioni di **code review** con il tutor. Tale pratica ha garantito un costante controllo sulla qualità del codice, sulla leggibilità, sul rispetto delle convenzioni di stile e sulla correttezza funzionale prima di procedere all'integrazione definitiva.

2.4 Issue Tracking e Controllo di Versione

Per il monitoraggio delle attività, la pianificazione operativa e la gestione centralizzata del codice sorgente, sono stati adottati strumenti open-source moderni ed efficenti, perfettamente integrati nei flussi di lavoro aziendali:

- **Controllo di versione:** La gestione del codice è stata affidata a **Gitea**, che ha ospitato i repository del progetto. Il flusso di lavoro è stato modellato attraverso l'uso di branch dedicati per singola funzionalità o correzione (`feature/`), l'apertura sistematica di *pull request* mirate e la supervisione tramite code review. Questa organizzazione ha permesso di isolare le modifiche sperimentali, preservare la stabilità del ramo principale e mantenere una cronologia pulita e tracciabile di tutte le evoluzioni del software.
- **Issue Tracking:** La tracciabilità e la pianificazione delle attività giornaliere e settimanali sono state gestite mediante la piattaforma **Plane**. Ogni requisito funzionale o correzione di bug è stato censito sotto forma di *issue*, suddiviso in task granulari e associato a specifiche etichette di priorità. Questo strumento ha consentito di monitorare visivamente lo stato di avanzamento delle 300 ore di stage, offrendo un riscontro immediato sul carico di lavoro corrente e facilitando la rendicontazione delle attività svolte.

2.5 Analisi preventiva dei rischi

Prima dell'avvio dello sviluppo, sono stati identificati i principali fattori di rischio associati all'hardware e all'integrazione software, definendo contestualmente le relative strategie di mitigazione:

- **Complessità di integrazione hardware-software e concorrenza:**
- **Descrizione:** La gestione simultanea del rendering grafico avanzato sul display e delle comunicazioni di rete asincrone con le API del CRM genera colli di bottiglia o blocchi del microcontrollore.

- **Mitigazione:** Adozione di un'architettura software strutturata in task paralleli, separando la logica di rete dalle routine di aggiornamento grafico e sincronizzando l'accesso alle risorse condivise in modo sicuro.
- **Probabilità:** Media
- **Impatto:** Alto

- **Esaurimento della memoria RAM e instabilità di rete:**
- **Descrizione:** Si verificano errori a runtime di tipo *out of memory* dovuti ai limiti fisici della memoria interna dell'ESP32, accompagnati da interruzioni operative prolungate in caso di instabilità o assenza della connessione Wi-Fi.
- **Mitigazione:** Ottimizzazione dell'uso della memoria RAM attraverso il rilascio mirato dei buffer grafici non utilizzati e implementazione della cache locale per garantire la continuità d'uso in caso di disconnessione temporanea della rete.
- **Probabilità:** Media
- **Impatto:** Medio-Alto

- **Evoluzione o instabilità delle API del CRM:**
- **Descrizione:** Modifiche impreviste agli endpoint o ai contratti delle API del sistema CRM esterno compromettono la corretta ricezione e trasmissione dei dati.
- **Mitigazione:** Astrazione del layer di comunicazione di rete tramite interfacce modulari e implementazione di robusti controlli di validazione e gestione delle eccezioni sui payload in ingresso e in uscita.
- **Probabilità:** Bassa
- **Impatto:** Alto

- **Prestazioni e fluidità dell'interfaccia grafica:**
- **Descrizione:** La visualizzazione di informazioni aggiornate in tempo reale e la gestione delle animazioni dell'interfaccia provocano rallentamenti o riducono la reattività del sistema sui dispositivi ESP32.

- **Mitigazione:** Ottimizzazione dei componenti grafici, riduzione degli aggiornamenti non necessari e monitoraggio dell'utilizzo delle risorse durante le attività di sviluppo e test.
- **Probabilità:** Media
- **Impatto:** Medio
- **Disallineamento dei requisiti funzionali per i pannelli:**
- **Descrizione:** Interpretazioni errate delle specifiche dei pannelli informativi o delle modalità di interazione con i display vincolati da risorse limitate comportano lo sviluppo di funzionalità non conformi alle reali esigenze operative dell'organizzazione cliente.
- **Mitigazione:** Gestione costante attraverso gli incontri periodici e regolari di allineamento e le sessioni di revisione con il tutor aziendale.
- **Probabilità:** Bassa
- **Impatto:** Medio

- **Sforamento dei tempi di sviluppo:**
- **Descrizione:** La curva di apprendimento legata alla programmazione embedded su ESP32, unita a criticità impreviste nell'interfacciamento con le API di RelAi, allunga i tempi di completamento delle attività previste dal cronoprogramma.
- **Mitigazione:** Pianificazione incrementale delle attività, definizione delle priorità in funzione dei requisiti obbligatori e monitoraggio costante dell'avanzamento mediante incontri periodici con il tutor aziendale, così da individuare tempestivamente eventuali ritardi.
- **Probabilità:** Media
- **Impatto:** Medio-Alto

3 Analisi dei requisiti

Il presente capitolo è dedicato all'analisi dei requisiti del sistema, attività essenziale per definire gli obiettivi funzionali e i vincoli progettuali della soluzione. Vengono illustrati gli attori coinvolti, i casi d'uso e i requisiti funzionali e non funzionali individuati durante la fase di studio. Le informazioni raccolte costituiscono il punto di partenza per le successive attività di progettazione e implementazione del sistema.

3.1 Analisi degli utenti

Il sistema sviluppato nel contesto dello stage consiste in un dispositivo basato su ESP32, dotato di display e integrato con il CRM RelAi. Il suo scopo è mostrare in tempo reale informazioni relative a organizzazioni ed eventi, quali codici QR, contatori degli iscritti e dei partecipanti, prenotazioni e dati di stato del dispositivo.
L'utilizzo del sistema coinvolge un unico attore principale, identificato nell'utente dell'organizzazione cliente presso la quale il dispositivo viene installato. Tale soggetto è responsabile della configurazione iniziale del pannello, che comprende il collegamento alla rete Wi-Fi tramite portale captive e la gestione delle principali impostazioni operative del dispositivo trammite CRM RelAi.
Una volta completata la fase di configurazione, l'utente interagisce con il sistema attraverso la visualizzazione ciclica o manuale dei pannelli informativi a display e l'utilizzo di un menu diagnostico touch locale per operazioni di manutenzione (riavvio, reset, condivisione QR). Parallelamente, il sistema dialoga con il cloud per ricevere aggiornamenti in tempo reale tramite WebSocket.
Poiché il progetto è destinato a contesti operativi nei quali gli utilizzatori potrebbero non possedere particolari competenze tecniche, l'interfaccia e i flussi di interazione sono stati progettati privilegiando chiarezza, semplicità di utilizzo e rapidità delle operazioni di configurazione e gestione. Tale approccio consente di ridurre la complessità percepita e di facilitare l'adozione del dispositivo presso le organizzazioni clienti.

3.2 Casi d'uso

Per lo studio dei casi di utilizzo del prodotto sono stati definiti i diagrammi e le relative specifiche funzionali. I diagrammi dei casi d'uso (*Use Case Diagram*) sono diagrammi di tipo *UML* dedicati alla descrizione delle funzioni o servizi offerti dal sistema, così come sono percepiti dagli attori che interagiscono con esso.

3.2.1 Elenco dei casi d'uso

- **UC1** – Configurazione Wi-Fi
  - **UC1.1** – Nessuna rete disponibile
  - **UC1.2** – Credenziali non valide
- **UC2** – Consultazione diagnostica locale del dispositivo
- **UC3** – Consultazione stato e telemetria remota tramite CRM RelAi
- **UC4** – Condivisione Wi-Fi tramite QR Code
- **UC5** – Riavvio locale del dispositivo
- **UC6** – Riavvio remoto del dispositivo
- **UC7** – Reset di sistema locale
- **UC8** – Reset di sistema remoto
- **UC9** – Sincronizzazione e gestione remota tramite CRM RelAi
  - **UC9.1** – Gestione del fallback in assenza di connettività
- **UC10** – Navigazione tra le schermate informative
- **UC11** – Visualizzazione dei dati nei pannelli informativi a display

3.2.2 Descrizione dettagliata dei casi d'uso

UC 1: Configurazione della connessione Wi-Fi

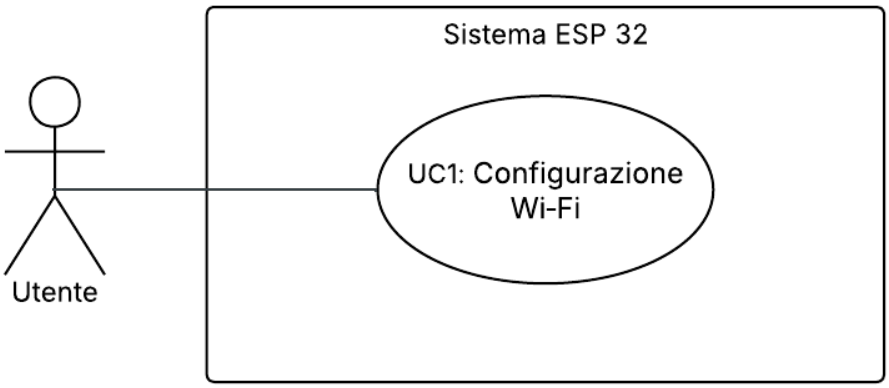


Figura 3.1: Diagramma del caso d'uso UC1: Configurazione della connessione Wi-Fi

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- È disponibile almeno una rete Wi-Fi nelle vicinanze.

Trigger: L'utente desidera configurare o aggiornare i parametri di accesso alla rete Wi-Fi del dispositivo.
Descrizione:

1. L'utente si connette all'Access Point generato dal dispositivo.
2. Il sistema reindirizza automaticamente l'utente alla pagina di configurazione.
3. Il sistema esegue la scansione delle reti Wi-Fi disponibili.
4. L'utente seleziona la rete desiderata.
5. L'utente inserisce la password della rete selezionata.
6. L'utente conferma la configurazione.
7. Il sistema verifica i dati ricevuti.
8. Il sistema salva le credenziali nella memoria persistente.
9. Il sistema effettua la connessione alla rete Wi-Fi selezionata.
10. Il sistema conferma l'avvenuta configurazione.

Postcondizioni:

- Le credenziali della rete Wi-Fi sono memorizzate nella memoria persistente del dispositivo.
- Il dispositivo risulta connesso alla rete selezionata.
- Il dispositivo è pronto per comunicare con le API del CRM RelAi.

UC 1.1: Nessuna rete disponibile

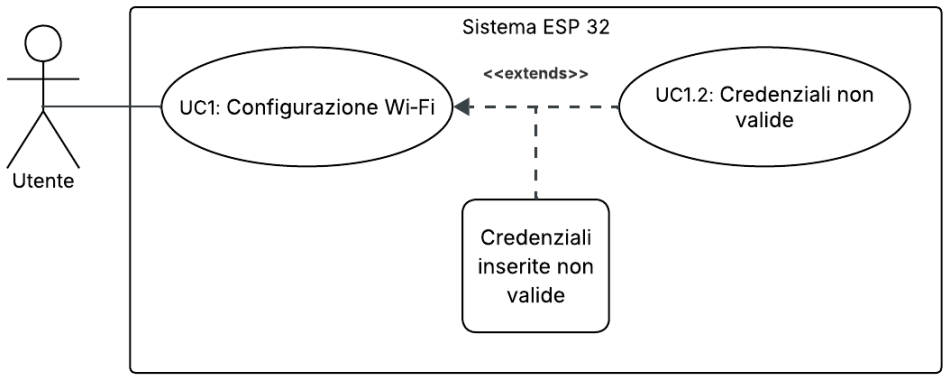


Figura 3.2: Diagramma del caso d'uso UC1.1: Nessuna rete disponibile

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è acceso con il Captive Portal attivo
- L'utente è connesso alla rete generata dal dispositivo.

Trigger: Avvio della scansione delle reti Wi-Fi senza la presenza di punti d'accesso validi nel raggio d'azione.
Descrizione:
1. L'utente accede alla pagina di configurazione Wi-Fi.
2. Il sistema avvia la scansione delle reti disponibili.
3. La scansione non restituisce alcuna rete utilizzabile o va in timeout.
4. Il sistema restituisce l'elenco vuoto e un feedback visivo d'errore all'interfaccia web.
5. L'utente richiede una nuova scansione.

Postcondizioni:
- Nessuna configurazione Wi-Fi viene salvata.
- Il Captive Portal e l'Access Point rimangono disponibili per un nuovo tentativo.

UC 1.2: Credenziali non valide

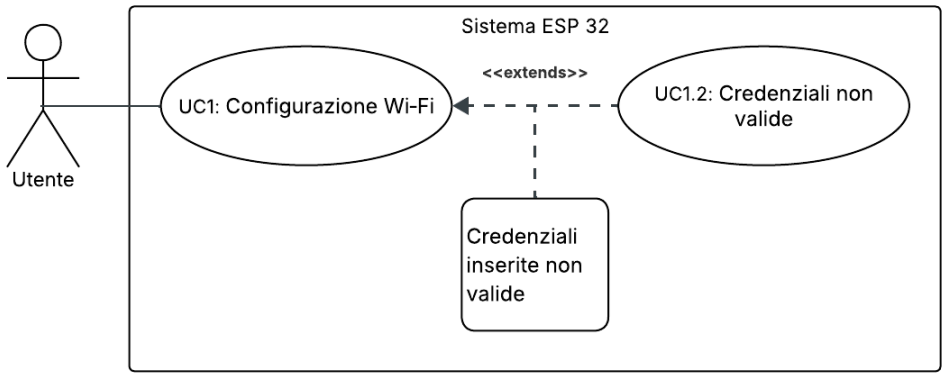


Figura 3.3: Diagramma del caso d'uso UC1.2: Credenziali non valide

Attori principali: Utente
Precondizioni:
- È stata selezionata una rete Wi-Fi.
- Sono stati inseriti dati errati.

Trigger: Tentativo di connessione a una rete Wi-Fi con parametri sbagliati.
Descrizione:
1. L'utente conferma la configurazione inserendo la password.
2. Il sistema tenta di collegarsi alla rete Wi-Fi per un intervallo massimo di timeout.
3. Il tentativo di associazione fallisce (codice di errore Wi-Fi).

4. Il sistema intercetta il fallimento, interrompe la connessione STA e ripristina la stabilità dell'Access Point.
5. Il sistema restituisce un messaggio di errore JSON all'interfaccia di provisioning.
6. L'utente corregge i parametri e ripete la procedura.

Postcondizioni:
- Nessuna credenziale errata viene salvata come configurazione stabile.
- Il portale di configurazione rimane attivo per consentire la correzione dei dati.

UC 2: Consultazione diagnostica locale del dispositivo

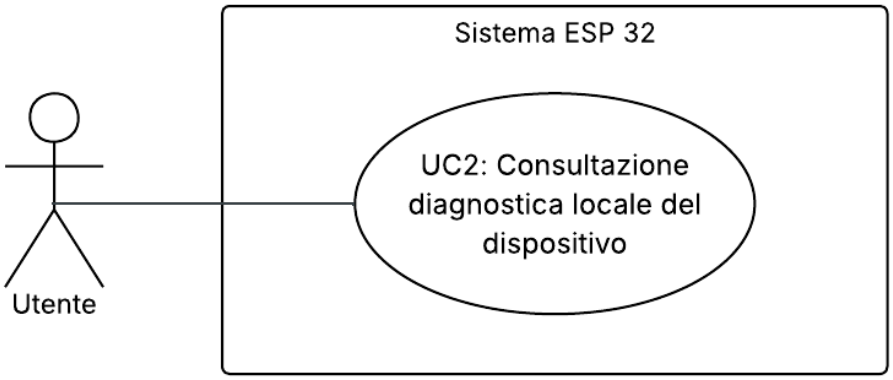


Figura 3.4: Diagramma del caso d'uso UC2: Consultazione diagnostica locale del dispositivo

Attori principali: Utente
Precondizioni:
- Il dispositivo ESP32 è operativo.
- L'interfaccia grafica è operativo.

Trigger: L'utente seleziona il pulsante «Stato Sistema» dal menu touch locale.
Descrizione:
1. L'utente preme il pulsante di accesso allo stato dal menu touch del display.
2. Il sistema interroga i moduli di runtime e raccoglie i parametri correnti (SSID della rete connessa, stato della connessione, versione del firmware e tempo trascorso *uptime* dall'ultima disconnessione).
3. Il sistema apre una *message box* contenente le informazioni aggregate.
4. L'utente prende visione dei dati diagnostici e chiude la finestra modale tramite il pulsante di chiusura dedicato, ripristinando la schermata principale.

Postcondizioni:

- L'utente è informato sullo stato di salute e connettività locale del dispositivo.
- Il sistema riprende la normale rotazione o visualizzazione dei pannelli alla chiusura del modale.

UC 3: Consultazione stato e telemetria remota tramite CRM RelAi

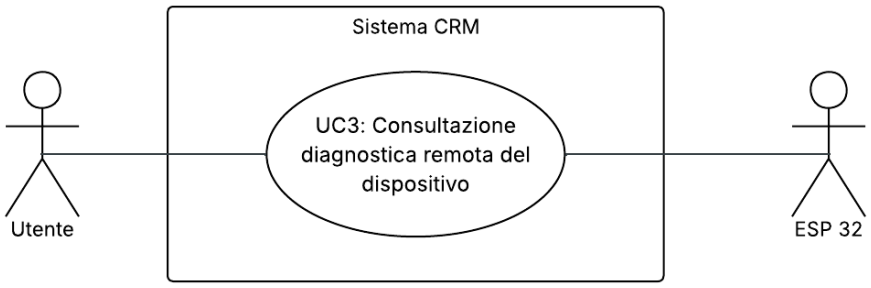


Figura 3.5: Diagramma del caso d'uso UC3: Consultazione stato e telemetria remota tramite CRM RelAi

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 è connesso a Internet tramite Wi-Fi.
- Il dispositivo ESP32 e connesso al CRM RelAi .

Trigger: L'utente desidera verificare lo stato di salute, la connettività o le metriche di un dispositivo dal pannello di controllo cloud.
Descrizione:

1. L'utente accede alla sezione di monitoraggio dei dispositivi all'interno della piattaforma CRM RelAi.
2. Il sistema cloud interroga i dati di telemetria più recenti memorizzati dal dispositivo (tra cui versione software, *uptime* e RSSI del Wi-Fi).
3. L'interfaccia della piattaforma CRM elabora le informazioni e mostra all'utente la panoramica aggiornata sullo stato di salute e connettività del display.

Postcondizioni:

- L'utente ottiene una visione centralizzata e aggiornata dello stato operativo del dispositivo da remoto.
- La piattaforma cloud mantiene traccia delle metriche telemetriche ricevute.

UC 4: Condivisione Wi-Fi tramite QR Code

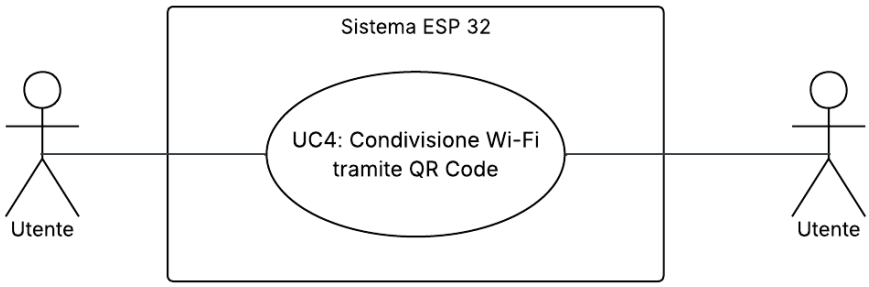


Figura 3.6: Diagramma del caso d'uso UC4: Condivisione Wi-Fi tramite QR Code

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è acceso ed è connesso a una rete Wi-Fi.

Trigger: L'utente seleziona il pulsante «QR Rete» dal menu di diagnostica locale per contividere i dati di accesso alla rete .
Descrizione:

1. L'utente accede al menu sul display e seleziona la condivisione della rete.
2. Il sistema recupera l'SSID e la chiave di rete.
3. Il sistema formatta la stringa secondo lo standard di connessione mobile.
4. Il sistema genera dinamicamente un codice QR e lo renderizza a schermo all'interno di una *message box* edicata.
5. Un dispositivo esterno scansiona il codice QR per connettersi rapidamnte alla medesima rete.
6. L'utente chiude il modale al termine dell'operazione.

Postcondizioni:

- Il codice QR con le credenziali correnti risulta visibile a schermo.
- Alla chiusura del modale, viene ripristinata la visualizzazione operativa standard e riattivato il timer di rotazione.

UC 5: Riavvio locale del dispositivo

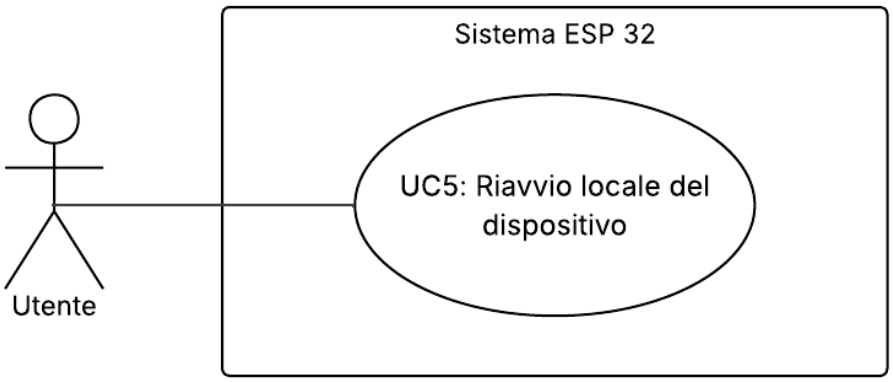


Figura 3.7: Diagramma del caso d'uso UC5: Riavvio locale del dispositivo

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- L'interfaccia grafica è operativo.

Trigger: L'utente seleziona il tasto «Riavvio ESP32» sul display.
Descrizione:

1. L'utente seleziona l'opzione di riavvio dal menu locale touch.
2. Il sistema avvia un timer temporizzato di sicurezza per la chiusura ordinata dei task.
3. Il sistema esegue il comando software di riavvio.
4. Il microcontrollore si riavvia ricaricando il firmware e ripristinando le periferiche e le connessioni di rete.

Postcondizioni:

- Il dispositivo viene riavviato pulitamente preservando le configurazioni e le preferenze salvate in memoria persistente NVS.

UC 6: Riavvio remoto del dispositivo

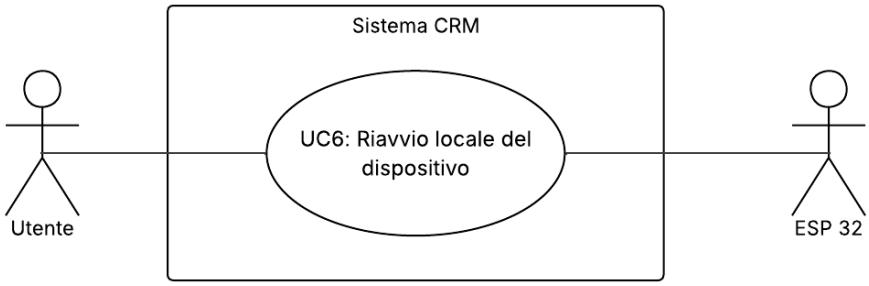


Figura 3.8: Diagramma del caso d'uso UC6: Riavvio remoto del dispositivo

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 è connesso a Internet tramite Wi-Fi.
- Il dispositivo ESP32 e connesso al CRM RelAi .

Trigger: L'utente invia un comando di riavvio remoto dalla piattaforma cloud CRM RelAi.
Descrizione:

1. Il sistema riceve un payload JSON tramite WebSocket contenente il comando di riavvio.
2. Il sistema elabora il comando remoto e avvia la procedura di chiusura delle attività in corso.
3. Il microcontrollore esegue il comando di riavvio software.
4. Il dispositivo si riavvia da remoto ricaricando il firmware.

Postcondizioni:

- Il dispositivo viene riavviato da remoto preservando le configurazioni salvate in memoria persistente NVS.

UC 7: Reset di sistema locale

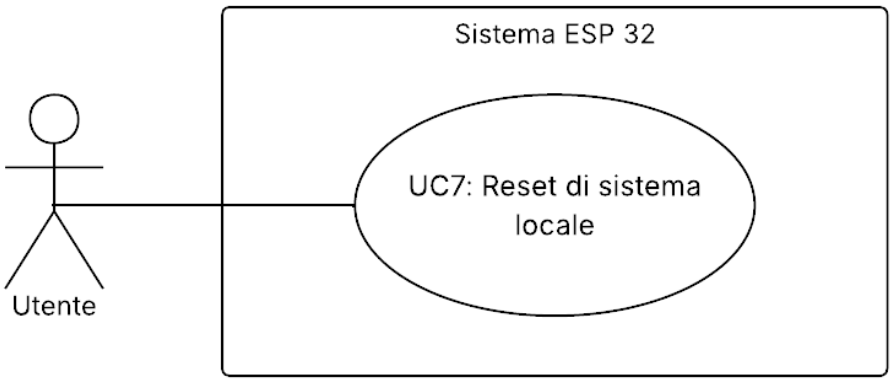


Figura 3.9: Diagramma del caso d'uso UC7: Reset di sistema locale

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è acceso e operativo.

Trigger: L'utente seleziona il tasto «Reset ESP32» dal display.
Descrizione:

1. L'utente seleziona l'opzione di reset di fabbrica dal menu locale.
2. Il sistema avvia la procedura di cancellazione dei namespace di configurazione nella memoria flash.
3. Il sistema disconnette le interfacce di rete e azzera le preferenze salvate.
4. Il sistema esegue il riavvio automatico del microcontrollore.
5. Al nuovo avvio, il dispositivo si presenta nello stato iniziale di fabbrica con l'Access Point di provisioning attivo.

Postcondizioni:

- Tutti i dati salvati, le preferenze di rete e la cache locale risultano completamente cancellati.
- Il dispositivo torna allo stato di configurazione iniziale pronto per una nuova associazione.

UC 8: Reset di sistema remoto

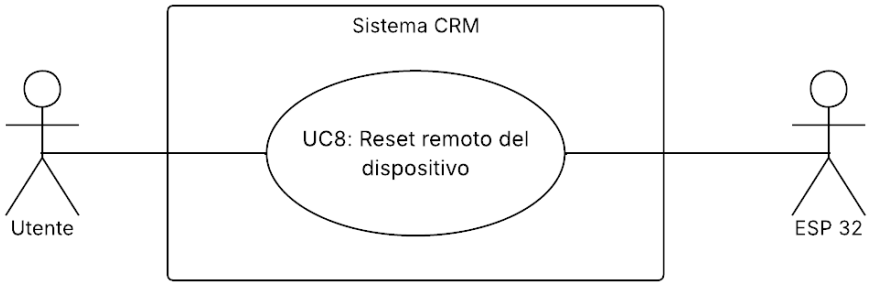


Figura 3.10: Diagramma del caso d'uso UC8: Reset di sistema remoto

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 è connesso a Internet tramite Wi-Fi.
- Il dispositivo ESP32 e connesso al CRM RelAi .

Trigger: L'utente invia un comando di reset di fabbrica dalla piattaforma cloud CRM RelAi.
Descrizione:

1. Il sistema riceve un payload JSON tramite WebSocket contenente il comando di reset.
2. Il sistema avvia la procedura di cancellazione remota dei dati nei namespace di configurazione presenti nella memoria flash.
3. Il sistema ripulisce le preferenze e interrompe le interfacce di rete.
4. Il sistema esegue il riavvio automatico del dispositivo portandolo allo stato iniziale di fabbrica.

Postcondizioni:

- Tutte le preferenze e i dati memorizzati in precedenza risultano cancellati da remoto.
- Il dispositivo torna allo stato di configurazione iniziale con l'Access Point di provisioning attivo.

UC 9: Aggiornamento remoto dei dati tramite CRM RelAi

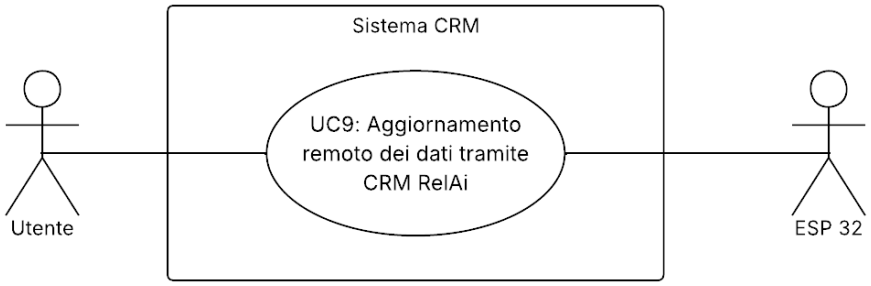


Figura 3.11: Diagramma del caso d'uso UC9: Aggiornamento remoto dei dati tramite CRM RelAi

Attori principali: Utente

Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 è connesso a Internet tramite Wi-Fi.
- Il dispositivo ESP32 e connesso al CRM RelAi .

Trigger: Ricezione di un evento di aggiornamento in tempo reale sul canale WebSocket.

Descrizione:

1. Il sistema riceve i dati di configurazione aggiornati e la lista aggiornata degli eventi dal server.
2. Il modulo di gestione dati elabora il payload JSON ricevuto e verifica se vi sono variazioni rispetto alla cache corrente.
3. Se i dati differiscono, il sistema salva i nuovi eventi nella memoria persistente NVS.
4. Il sistema invia un segnale interno di aggiornamento per ricostruire dinamicamente le schermate informative.

Postcondizioni:

- Le schermate informative, i dati degli eventi e i parametri di rotazione si aggiornano dinamicamente in base alle direttive del CRM RelAi.

UC 9.1: Gestione del fallback in assenza di connettività

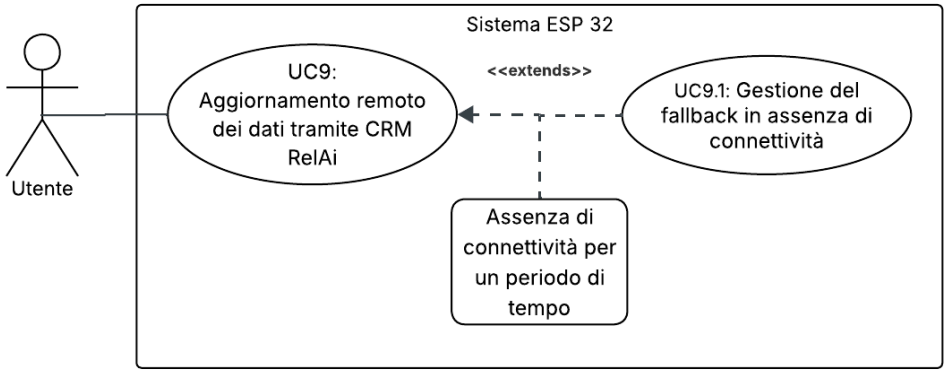


Figura 3.12: Diagramma del caso d'uso UC9.1: Gestione del fallback in assenza di connettività

Attori principali: Utente
Precondizioni:

- Il dispositivo si trova in uno stato di assenza temporanea di connessione della rete o il server CRM RelAi risulta irragiungibile.

Trigger: Fallimento di una richiesta di aggiornamento dei dati in tempo reale.
Descrizione:

1. Il sistema rileva il fallimento della comunicazione di rete o l'assenza prolungata di connettività.
2. Il gestore delle API intercetta l'eccezione bloccando i tentativi non necessari e prevenendo la frammentazione della memoria RAM.
3. Il sistema legge in modo trasparente i dati precedentemente memorizzati nella cache persistente locale.
4. L'interfaccia grafica continua a mostrare l'ultimo stato valido degli eventi senza causare blocchi o riavvii anomali del display.
5. Il sistema avvia una procedura di tentativi schedulati di riconnessione in background.

Postcondizioni:

- Il dispositivo garantisce la continuità operativa in modalità *offline* (fallback) sfruttando la cache locale.
- Il sistema riprende la sincronizzazione automatica non appena la connettività di rete viene ristabilita.

UC 10: Navigazione tra le schermate informative

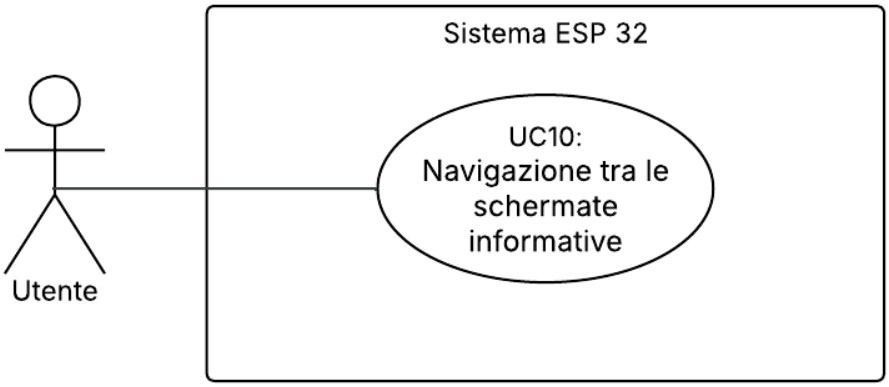


Figura 3.13: Diagramma del caso d'uso UC10: Navigazione tra le schermate informative

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 ha dei dati salvati da mostrare.

Trigger: Pressione dei pulsanti touch di navigazione o scadenza del timer di rotazione automatica; triplo tocco sullo schermo per abilitare/disabilitare la rotazione.
Descrizione:

1. Il sistema gestisce la rotazione automatica delle pagine in base al timer configurato (default 30 secondi, impostato remoto da CRM RelAi), oppure l'utente interagisce premendo i pulsanti freccia sulla barra di navigazione.
2. Il sistema esegue l'animazione di transizione tra la schermata corrente e la successiva.
3. L'utente visualizza la nuova schermata informativa o la pagina del menu diagnostico.

Postcondizioni:

- L'interfaccia grafica risponde fluidamente ai comandi di navigazione manuale o automatica.
- Lo stato della rotazione (attiva o in pausa) viene aggiornato visivamente sui pulsanti di controllo.

UC 11: Visualizzazione dei dati nei pannelli informativi a display

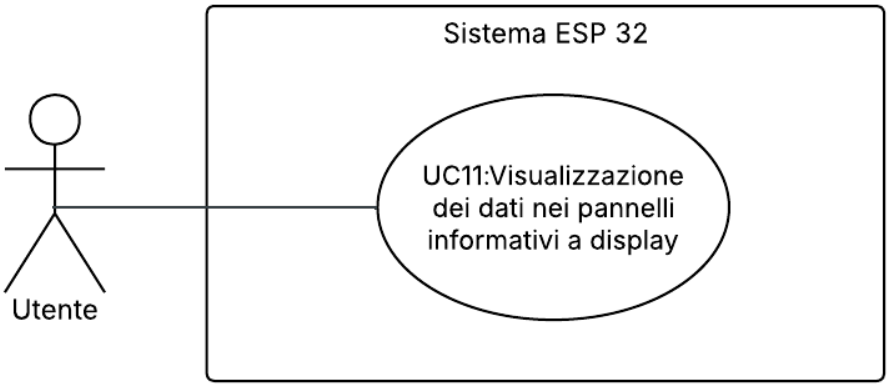


Figura 3.14: Diagramma del caso d'uso UC11: Visualizzazione dei dati nei pannelli informativi a display

Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 ha dei dati salvati da mostrare.

Trigger: Completamento dell'avvio del dispositivo e ricezione di nuovi dati.
Descrizione:

1. Il sistema esegue il rendering grafico dei pannelli degli eventi strutturati a schede.
2. Ciascun pannello mostra a schermo le informazioni principali dell'evento (nome dell'evento, contatore delle prenotazioni e dei partecipanti).
3. Il sistema genera e mostra la tile laterale associata contenente il codice QR dinamico di riferimento per l'evento.
4. L'utente consulta visivamente le informazioni o interagisce con il touch screen.

Postcondizioni:

- I dati aggiornati dell'evento risultano correttamente renderizzati e visibili a schermo con elevata fluidità visiva.
- Il sistema mantiene la rotazione attiva in base alle impostazioni correnti.

3.3 Requisiti del sistema e stato di completamento

La definizione dei requisiti costituisce il fondamento dell'intero processo di progettazione del sistema. A partire dall'analisi del contesto aziendale, delle esigenze dei clienti e delle criticità tecniche legate all'hardware ESP32, sono stati individuati i requisiti funzionali, non funzionali e di vincolo necessari a

garantire la corretta operatività del prototipo. L'obiettivo principale è stato la realizzazione di una soluzione in grado di mostrare informazioni rilevanti in modo affidabile, semplice e immediatamente leggibile, mantenendo un elevato livello di robustezza anche in condizioni di rete instabile o in presenza di aggiornamenti intermittenti da parte del sistema centrale.

In fase di sviluppo, tutti i requisiti individuati sono stati verificati in modo diretto e progressivo, con riferimento ai casi d'uso precedentemente definiti, alle attività di integrazione con RelAi e ai test eseguiti sul dispositivo. L'esito di questa valutazione ha confermato la completezza del sistema rispetto alle esigenze inizialmente dichiarate. Di conseguenza, il livello di copertura dei requisiti può essere considerato pienamente soddisfatto.

3.3.1 Requisiti funzionali e stato di attuazione

I requisiti funzionali definiscono le capacità che il sistema deve mettere a disposizione dell'organizzazione cliente e dell'utente finale. Essi riguardano in particolare la configurazione del dispositivo, la lettura dei dati di evento, la visualizzazione dei contenuti sul display e la gestione delle operazioni di manutenzione.

Lo stato di attuazione dei requisiti funzionali è riepilogato nella Tabella 3.1.

Codice	**Requisito**	**Stato**
RF01	Configurazione iniziale della connessione Wi-Fi tramite portale locale e salvataggio persistente delle credenziali.	Completato
RF02	Abbinamento del dispositivo a RelAi e ricezione delle informazioni di configurazione e degli eventi.	Completato
RF03	Visualizzazione sul display dei pannelli informativi relativi a iscritti, partecipanti, prenotazioni e codice QR.	Completato
RF04	Rotazione automatica e navigazione manuale tra le schermate informative.	Completato
RF05	Diagnostica locale del dispositivo con informazioni sullo stato della rete, dell'uptime e del firmware.	Completato

Codice	Requisito	Stato
RF06	Operazioni di manutenzione come riavvio, reset e condivisione della rete tramite QR Code.	Completato
RF07	Gestione di fallback e cache locale in caso di assenza o instabilità della rete.	Completato
RF08	Configurazione remota dei pannelli e della frequenza di aggiornamento da parte del CRM RelAi.	Completato

Tabella 3.1: Requisiti funzionali e stato di completamento

3.3.2 Requisiti non funzionali, di qualità e vincolo

I requisiti non funzionali descrivono le caratteristiche di qualità che un sistema embedded deve possedere per essere efficiente, affidabile e facilmente gestibile anche in contesti operativi reali. Tra questi, le criticità più rilevanti riguardano la stabilità del firmware, la reattività dell'interfaccia, l'usabilità del processo di configurazione e la sostenibilità del sistema sul lungo periodo.

I requisiti non funzionali e di vincolo sono riportati nella Tabella 3.2.

Codice	Requisito	Stato
RNF01	Affidabilità del sistema in presenza di disconnessioni temporanee e timeout nelle chiamate verso il CRM.	Completato
RNF02	Robustezza del firmware con gestione degli errori, riconnessione automatica e recupero dalla cache locale.	Completato
RNF03	Prestazioni adeguate per il rendering delle schermate e aggiornamento dei dati in tempo reale.	Completato
RNF04	Usabilità delle operazioni locali per utenti non esperti, con procedure di approccio semplici e trasparenti.	Completato
RNF05	Manutenibilità del codice e separazione dei moduli per facilitare aggiornamenti e test futuri.	Completato

Codice	Requisito	Stato
RV01	Esecuzione del sistema su piattaforma ESP32 con vincoli di memoria, potenza computazionale e rendering.	Completato
RV02	Integrazione con l'ecosistema RelAi e compatibilità con i dati e i protocolli aziendali.	Completato
RV03	Possibilità di configurazione del dispositivo anche senza accesso a internet tramite modalità locale.	Completato

Tabella 3.2: Requisiti non funzionali e di vincolo

3.3.3 Conclusione sulla soddisfazione dei requisiti

Dall'analisi condotta durante lo stage emerge che tutti i requisiti inizialmente individuati sono stati soddisfatti in modo completo, sia nei loro aspetti funzionali sia in quelli di qualità e di vincolo. La soluzione ottenuta dimostra quindi la validità dell'approccio progettuale adottato, confermando che il dispositivo embedded può essere integrato efficacemente con il CRM RelAi e utilizzare le risorse hardware disponibili in modo sostenibile ed efficiente.

In particolare, il sistema raggiunge l'obiettivo di trasformare dati aziendali complessi in informazioni facilmente leggibili e accessibili direttamente presso le sedi degli utenti, riducendo l'interazione richiesta e migliorando la fruibilità dell'informazione. La raggiunta completezza dei requisiti rappresenta inoltre una base solida per eventuali sviluppi futuri, come nuove tipologie di pannelli, aggiornamenti OTA o integrazioni con flussi dati ancora più avanzati.

4 Tecnologie utilizzate

Questo capitolo descrive le principali tecnologie hardware e software adottate per realizzare il prototipo di interfaccia informativa integrato con il CRM RelAi. La scelta dei componenti è stata guidata dai requisiti individuati: disponibilità di un display interattivo, possibilità di comunicare tramite rete Wi-Fi, compatibilità con un ambiente di sviluppo per microcontrollori e capacità di aggiornare i dati ricevuti dal sistema centrale.
La soluzione è stata sviluppata in C++ per microcontrollori della famiglia ESP32-S3, utilizzando la libreria grafica LVGL e l'ambiente di sviluppo PlatformIO. Per la comunicazione con il CRM RelAi è stato impiegato un canale WebSocket; la configurazione iniziale della rete avviene invece attraverso un Access Point generato dal dispositivo e un portale di provisioning locale.

4.1 Piattaforma hardware

Il dispositivo principale del prototipo è basato su ESP32-S3 ed è abbinato a un display rotondo da 1,75 pollici (Figura 4.1). Questa configurazione è stata scelta per realizzare un pannello compatto, dotato di interazione touch e adatto alla visualizzazione delle informazioni sintetiche previste dai casi d'uso, come contatori, stato del dispositivo e codici QR.
In coerenza con il requisito facoltativo di supporto a più tipologie di display, è stata inoltre utilizzata una seconda scheda ESP32 con display quadrato da 1,8 pollici (Figura 4.2). La disponibilità di due formati consente di verificare l'adattabilità dell'interfaccia a geometrie differenti e costituisce un primo passo verso il supporto di ulteriori dispositivi. Le due configurazioni condividono la piattaforma software grafica, mentre la gestione delle periferiche dipende dalle caratteristiche specifiche della scheda e del display.
Il microcontrollore gestisce l'esecuzione del firmware, la connettività Wi-Fi, l'elaborazione dei dati e l'aggiornamento dell'interfaccia. Le risorse limitate rispetto a un sistema desktop richiedono attenzione nell'uso della memoria e nella frequenza di ridisegno, soprattutto durante la visualizzazione di elementi grafici come codici QR e animazioni.



Figura 4.1: Scheda ESP32-S3 con display rotondo da 1,75 pollici utilizzata come dispositivo principale



Figura 4.2: Scheda ESP32-S3 con display quadrato da 1,8 pollici utilizzata per verificare il requisito facoltativo multi-display

4.2 Linguaggio C++

Il firmware è stato scritto in C++ (Figura 4.3), linguaggio che permette di organizzare il programma in componenti riutilizzabili e di interagire con le

periferiche e le librerie disponibili per la piattaforma ESP32. Il codice coordina la gestione della rete, la ricezione e l'elaborazione dei dati, la logica di navigazione e il disegno dei pannelli sul display.
L'impiego di C++ consente di integrare la logica applicativa con le librerie per la grafica, la connettività e la gestione delle periferiche. La struttura del firmware deve tenere conto delle risorse limitate del microcontrollore e coordinare in modo efficiente l'elaborazione dei dati, gli aggiornamenti dell'interfaccia e le operazioni di rete.



Figura 4.3: Logo del linguaggio di programmazione C++

4.3 PlatformIO

Per configurare e gestire il progetto è stato utilizzato PlatformIO (Figura 4.4), ambiente di sviluppo integrato con Visual Studio Code. Lo strumento permette di dichiarare la piattaforma hardware di destinazione e le dipendenze del firmware, oltre a gestire le operazioni di compilazione e caricamento del programma sulla scheda.
La configurazione del progetto raccoglie i parametri necessari a riprodurre la toolchain e distingue le impostazioni comuni del firmware da quelle legate alle singole schede. PlatformIO è stato inoltre utilizzato nel ciclo iterativo di sviluppo: dopo ogni modifica, il firmware viene compilato e caricato sul dispositivo per verificare il comportamento effettivo dell'interfaccia e delle funzioni di rete.



Figura 4.4: Logo dell'ambiente di sviluppo PlatformIO

4.4 LVGL e interfaccia grafica

Per la realizzazione dell'interfaccia grafica è stata utilizzata LVGL (*Light and Versatile Graphics Library*), libreria orientata a sistemi embedded con display. LVGL fornisce componenti visuali e strumenti per costruire schermate interattive, gestire gli eventi di input e aggiornare le aree dello schermo senza dover implementare da zero tutte le primitive grafiche. Un esempio dell'interfaccia ottenuta è riportato in Figura 4.5.
Nel progetto la libreria viene utilizzata per comporre i pannelli informativi, mostrare testo, contatori e codici QR, gestire gli elementi di navigazione e presentare le schermate diagnostiche. La separazione tra logica applicativa e componenti visuali facilita l'aggiornamento dei contenuti quando arrivano nuovi dati dal CRM e permette di mantenere coerente l'interazione sulle due tipologie di display.
L'integrazione con l'hardware richiede inoltre i driver adatti al pannello e al sistema di input touch della scheda. Le dimensioni e la forma dello schermo influenzano la disposizione degli elementi: i contenuti devono rimanere leggibili e accessibili sia nel formato rotondo sia in quello quadrato.



Figura 4.5: Esempio dell'interfaccia realizzata con LVGL sul display del dispositivo

4.5 Connettività e protocolli

La connettività wireless è basata su Wi-Fi e viene utilizzata sia durante la configurazione sia durante il funzionamento ordinario. Nella fase di provisioning il dispositivo opera in modalità Access Point (AP), creando una rete alla quale l'utente può collegarsi per aprire il portale locale e configurare le

credenziali della rete dell'organizzazione. Dopo la configurazione, il dispositivo opera in modalità station (STA), collegandosi al punto di accesso disponibile per raggiungere il CRM RelAi.
L'Access Point mantiene disponibile la procedura di configurazione quando il dispositivo non è ancora associato a una rete o quando le credenziali non sono valide. I casi d'uso prevedono il mantenimento del portale durante gli errori di scansione o connessione, così che l'utente possa correggere i parametri senza dover intervenire sul firmware. Il reset di fabbrica riporta il dispositivo allo stato iniziale e rende nuovamente disponibile la modalità di provisioning.
La comunicazione in tempo reale con il CRM RelAi utilizza WebSocket. A differenza di una sequenza di richieste indipendenti, il canale WebSocket consente di mantenere una connessione persistente attraverso la quale il dispositivo può ricevere aggiornamenti e comandi remoti, come la sincronizzazione dei dati, il riavvio e il reset. I messaggi applicativi descritti nei casi d'uso sono rappresentati in formato JSON; il firmware interpreta il contenuto e aggiorna i dati locali o lo stato del dispositivo.
Quando la connessione al CRM o alla rete non è disponibile, il sistema utilizza i dati validi conservati localmente e avvia tentativi di riconnessione. Questo comportamento consente ai pannelli di continuare a mostrare l'ultimo stato noto e limita l'impatto delle interruzioni temporanee sulla fruizione delle informazioni.

4.6 Persistenza e gestione dei dati

Le impostazioni necessarie al funzionamento del dispositivo e i dati utilizzati per il recupero dell'ultimo stato valido sono conservati in memoria non volatile. Nei casi d'uso la memoria persistente è identificata come NVS (*Non-Volatile Storage*): essa permette di mantenere le configurazioni anche in seguito a un riavvio. La procedura di reset elimina le informazioni previste e riporta il dispositivo alla fase di configurazione iniziale.
I dati ricevuti dal CRM vengono elaborati dal firmware e confrontati con quelli già disponibili. Quando è necessario, la copia locale viene aggiornata e l'interfaccia viene ridisegnata per riflettere i nuovi contenuti. La cache locale svolge inoltre la funzione di fallback: in assenza di connettività permette di continuare a visualizzare le informazioni dell'ultima sincronizzazione valida.

4.7 Controllo di versione e documentazione

Lo sviluppo e la documentazione richiedono strumenti che consentano di mantenere tracciabili le modifiche, organizzare i materiali del progetto e

produrre elaborati aggiornabili. A questo scopo sono stati utilizzati Gitea per il controllo di versione e Typst per la stesura della tesi.

4.7.1 Gitea

Gitea (Figura 4.6) è stato utilizzato come piattaforma di gestione dei repository del progetto firmware. Il controllo di versione permette di conservare la cronologia delle modifiche, isolare gli interventi in branch dedicati e sottoporre le modifiche a revisione tramite *pull request* e *code review*. Come descritto nel capitolo dedicato ai processi, questa organizzazione supporta la collaborazione con il tutor e facilita l'individuazione delle modifiche introdotte durante le diverse fasi di sviluppo.



Figura 4.6: Logo della piattaforma Gitea

4.7.2 Typst per la redazione della tesi

La documentazione finale è stata redatta in Typst (Figura 4.7), un sistema di composizione tipografica basato su markup. La tesi è suddivisa in file sorgente distinti per i capitoli e utilizza un file principale per configurare il documento e includere i contenuti. Questa struttura modulare rende più agevole aggiornare le singole parti e mantenere uniformi titoli, riferimenti, figure e tabelle.
La compilazione dei sorgenti produce il documento PDF destinato alla consultazione e alla consegna. L'uso di un formato testuale versionabile permette inoltre di tracciare le revisioni della documentazione insieme ai materiali del progetto e di rigenerare il PDF dopo gli aggiornamenti.



Figura 4.7: Logo del sistema di composizione Typst

4.8 Riepilogo dello stack tecnologico

La Tabella 4.1 riassume i principali elementi tecnologici impiegati e il ruolo svolto all'interno del prototipo.

Ambito	Tecnologia	Utilizzo nel progetto
Linguaggio	C++	Implementazione del firmware e della logica applicativa.
Microcontrollore	ESP32-S3 e scheda ESP32	Esecuzione del firmware e gestione delle periferiche; rispettivamente con display rotondo da 1,75 pollici e quadrato da 1,8 pollici.
Interfaccia grafica	LVGL	Composizione delle schermate, gestione dei componenti visuali e degli input touch.
Toolchain	PlatformIO	Configurazione del progetto, compilazione, caricamento e verifica del firmware.
Rete locale	Wi-Fi AP e STA	Provisioning locale tramite Access Point e collegamento del dispositivo alla rete dell'organizzazione in modalità station.
Comunicazione cloud	WebSocket e JSON	Ricezione di aggiornamenti e comandi dal CRM RelAi tramite messaggi applicativi.
Persistenza	NVS e cache locale	Conservazione delle configurazioni e dei dati necessari al ripristino dell'ultimo stato valido.
Controllo versione	Gitea	Gestione dei repository, cronologia delle modifiche, branch e revisioni del codice.
Documentazione	Typst	Redazione modulare della tesi e generazione del documento PDF.

Tabella 4.1: Riepilogo delle tecnologie utilizzate

Le tecnologie descritte costituiscono la base per le successive attività di progettazione e implementazione del firmware. In particolare, la piattaforma hardware e LVGL definiscono il contesto della presentazione grafica, mentre Wi-Fi, WebSocket e la persistenza locale consentono l'integrazione con RelAi e la continuità del servizio.

5 Progettazione e codifica

Il firmware è stato sviluppato per supportare due distinte varianti hardware basate su microcontrollore ESP32-S3: una configurazione dotata di display rotondo da 1,75 pollici e una versione equipaggiata con uno schermo da 1,8 pollici. Sebbene le due varianti condividano i moduli logici per la gestione dei dati e della connettività di rete, presentano implementazioni differenziate per quanto concerne i driver del display, la gestione del touch screen e le relative modalità di navigazione dell'interfaccia.
Al fine di garantire una trattazione organica e strutturata, il capitolo ripercorre il flusso logico ed elaborativo del sistema: si procede inizialmente con l'analisi della fase di avvio e della modularità del firmware, per poi approfondire i meccanismi di comunicazione con il CRM e la persistenza locale dei dati, concludendo con la descrizione delle interfacce dedicate alla consultazione e alla configurazione del sistema. Gli stralci di codice sorgente inclusi nel testo provengono dai progetti sviluppati; essi sono stati opportunamente sintetizzati ove necessario e depurati da informazioni sensibili o credenziali di autenticazione.

5.1 Struttura del firmware

L'architettura del software adotta un approccio modulare volto a disaccoppiare la gestione dell'hardware di I/O (display AMOLED e touch screen capacitivo) dai servizi di rete, dal server HTTP locale e dall'interazione in tempo reale con la piattaforma cloud RelAi. Le responsabilità del sistema sono ripartite nei seguenti moduli sorgente:

- **`main.cpp`**: punto d'ingresso principale, gestisce l'avvio delle periferiche e la pianificazione dei task FreeRTOS sui due core dell'ESP32-S3.
- **`display_config.cpp`**: gestisce l'inizializzazione del display AMOLED e del touch screen con implementazioni differenziate per le due varianti hardware (display rotondo da 1,75 pollici e schermo da 1,8 pollici), configurando il bus QSPI, i relativi controller I²C della famiglia CST e l'allocazione dei buffer grafici in PSRAM per il motore LVGL.
- **`interfaccia_ui.cpp`**: implementa la costruzione delle schermate dinamiche degli eventi adattate alle specifiche geometriche e di navigazione delle due varianti hardware (layout circolare vs rettangolare), la generazione dei QR code, la decodifica delle immagini PNG del logo tramite lodepng e la gestione delle modali di diagnostica e del menu.

- **`wifi_conessione.cpp`**: configura la connettività di rete in modalità duale AP+STA, il server HTTP locale (WebServer), il captive portal DNS e la gestione del file system LittleFS.
- **`api.cpp`**: governa la comunicazione bidirezionale con il CRM RelAi (heartbeat periodico, client WebSocket, sincronizzazione della cache eventi nella memoria flash non volatile NVS, scaricamento del logo e aggiornamento OTA).

5.2 Backend: avvio, task e accesso condiviso

La funzione `setup()` all'interno di `main.cpp` inizializza la seriale, il server locale, il display e LVGL, creando successivamente due task FreeRTOS dedicati e associati ai rispettivi core (Codice 5.1). I task sono creati con priorità 1 per la rete e 2 per il display:

```
1  void setup() {
2    Serial.begin(115200);
3    delay(1000);
4
5    Serial.println("\n--- Avvio ESP32-S3 ---");
6
7    inizializza_sistema_reti();
8
9    init_display_touch_lvgl();
10   ui_load_cached_data();
11
12   xTaskCreatePinnedToCore(networkTask, "NetworkTask", 24576, NULL, 1,
13                           &NetworkTaskHandle, 0);
14   xTaskCreatePinnedToCore(displayTask, "DisplayTask", 16384, NULL, 2,
15                           &DisplayTaskHandle, 1);
16
17   Serial.println("[SYSTEM] Avvio completato.");
18 }
```

Codice 5.1: Funzione `setup()` dell'ESP32-S3

- **`NetworkTask` (Core 0, priorità 0)**: gestisce il provisioning Wi-Fi, il server web permanente, le richieste HTTP, il client WebSocket con il CRM, il download del logo dell'organizzazione e la segnalazione all'utente di eventuali perdite di connettività.
- **`DisplayTask` (Core 1, priorità 1)**: esegue ciclicamente il rendering grafico e la gestione degli input tramite LVGL. La priorità numerica più alta assegna a questo task una precedenza maggiore rispetto al task di rete, senza costituire una garanzia assoluta di fluidità in presenza di operazioni bloccanti.

Poiché le risorse di LVGL non sono thread-safe, le operazioni grafiche sono protette mediante l'utilizzo del mutex lvgl_mutex, creato in init_display_touch_lvgl(). Il task del display acquisisce il semaforo prima di invocare l'handler dei timer grafici e ne limita il tempo di attesa tra 1 e 10 ms (Codice 5.2):

```
1  void displayTask(void *pvParameters) {
2    if (xSemaphoreTake(lvgl_mutex, portMAX_DELAY)) {
3      create_tileview();
4      xSemaphoreGive(lvgl_mutex);
5    }
6
7    while (true) {
8      uint32_t time_till_next = 5;
9
10     if (xSemaphoreTake(lvgl_mutex, pdMS_TO_TICKS(10))) {
11       time_till_next = lv_timer_handler();
12       xSemaphoreGive(lvgl_mutex);
13     }
14
15     if (time_till_next < 1) time_till_next = 1;
16     if (time_till_next > 10) time_till_next = 10;
17
18     vTaskDelay(pdMS_TO_TICKS(time_till_next));
19   }
20 }
```

Codice 5.2: Ciclo principale del task di rendering

Anche il `NetworkTask` utilizza lo stesso mutex ogni volta che deve interagire con l'interfaccia: aggiornamento delle etichette di stato ogni secondo, apertura e chiusura del messaggio di errore di rete (mostrato quando la connessione al router risulta assente da oltre 15 secondi, se è presente una rete salvata) e ricostruzione delle schermate. Quest'ultima avviene tramite un meccanismo a flag: i moduli che ricevono nuovi dati dal CRM invocano `ui_richiedi_ricarica_tileview()`, mentre il `NetworkTask` verifica periodicamente il flag con `ui_consuma_richiesta_ricarica()` e, se attivo, richiama `create_tileview()` mantenendo il mutex. In questo modo i thread di rete non manipolano mai direttamente gli oggetti grafici senza sincronizzazione.

5.3 Backend: provisioning Wi-Fi e server locale

All'avvio del `NetworkTask`, il dispositivo attiva la modalità `WIFI_AP_STA`, avviando contemporaneamente un Access Point locale (`ESP32-Config`, IP `192.168.4.1`) e il tentativo di connessione alla rete station eventualmente salvata nelle `Preferences`. Un server DNS locale (`DNSServer`) risponde a qual-

siasi dominio con l'indirizzo dell'Access Point, così da reindirizzare le richieste dei client verso il portale captive (Codice 5.3). L'Access Point rimane attivo in modo permanente, anche a connessione STA avvenuta, in modo da consentire in qualsiasi momento la riconfigurazione del dispositivo.

```
void avvia_wifi_provisioning() {
  server.stop();
  dnsServer.stop();

  WiFi.mode(WIFI_AP_STA);
  WiFi.setAutoReconnect(true);
  WiFi.setSleep(false);

  WiFi.softAPConfig(AP_IP, AP_IP, IPAddress(255, 255, 255, 0));
  if (!WiFi.softAP(AP_SSID, AP_PASS)) {
    Serial.println("[Wi-Fi AP] ERRORE: avvio Access Point fallito!");
  }
  dnsServer.start(DNS_PORT, "*", AP_IP);
  ...
}
```

Codice 5.3: Avvio dell'Access Point e del DNS captive (versione sintetizzata)

Le richieste il cui header `Host` non coincide con l'indirizzo dell'Access Point vengono ridirezionate con una risposta HTTP 302 verso `http://192.168.4.1/`; le URL di verifica della connettività dei sistemi operativi (ad esempio `generate_204`) ricevono invece una risposta 204. I file statici del portale sono letti da LittleFS, con supporto ai file compressi `.gz` serviti con l'header `Content-Encoding: gzip`.

Il server HTTP espone endpoint REST dedicati alla gestione e alla manutenzione del sistema, registrati in `inizializza_sistema_reti()` (Codice 5.4). Tra questi, `/scan` esegue la scansione asincrona delle reti disponibili, `/connect` verifica le credenziali ricevute tentando la connessione STA e le salva in caso di successo, `/status` restituisce lo stato corrente in formato JSON, mentre `/reset_wifi`, `/reboot` e `/reset_factory` consentono rispettivamente la cancellazione delle credenziali, il riavvio e il ripristino di fabbrica. La funzione `inizializza_api()` aggiunge infine l'endpoint `/api/events`, che restituisce la cache locale degli eventi.

```
void inizializza_sistema_reti() {
  if (!LittleFS.begin(true)) {
    Serial.println("[LittleFS] ERRORE: Montaggio fallito!");
  } else {
    Serial.println("[LittleFS] File system montato correttamente.");
  }

```

```
  server.on("/", HTTP_GET, handle_root);
  server.on("/scan", HTTP_GET, handle_scan);
  inizializza_api(server);

  server.on("/status", HTTP_GET, handle_get_status);
  server.on("/connect", HTTP_POST, handle_connect);
  server.on("/reset_wifi", HTTP_POST, handle_reset_wifi);
  server.on("/reboot", HTTP_POST, handle_reboot);
  server.on("/reset_factory", HTTP_POST, handle_reset_factory);
  server.onNotFound(handle_not_found);
}
```

Codice 5.4: Registrazione degli endpoint del server HTTP locale

La scansione delle reti è avviata in modalità asincrona. Il firmware disconnette temporaneamente la STA solo se non è già connessa; in tal caso, al termine della scansione tenta di ripristinare il collegamento usando le credenziali salvate. Il server viene gestito dal `NetworkTask` tramite `gestisci_wifi_prov()`, che elabora le richieste DNS e HTTP e invoca la gestione periodica del client WebSocket e dell'heartbeat.

5.4 Backend: sincronizzazione con RelAi

5.4.1 Heartbeat e telemetria

Il dispositivo invia periodicamente richieste HTTP POST (heartbeat) all'endpoint del CRM, autenticandosi con un token Bearer e trasmettendo dati di telemetria quali uptime, memoria heap libera e intensità del segnale RSSI (Codice 5.5). Il client TLS è condiviso tra heartbeat e autenticazione dei canali WebSocket, per riutilizzare la connessione (keep-alive) e ridurre il consumo di memoria.

```
static bool esegui_heartbeat() {
  WiFiClientSecure &client = https_client();
  HTTPClient http;
  http.setReuse(true);

  if (!http.begin(client, HEARTBEAT_URL)) {
    return false;
  }

  http.setConnectTimeout(3000);
  http.setTimeout(5000);
  http.addHeader("Authorization", String("Bearer ") + API_TOKEN);
  http.addHeader("Content-Type", "application/json");

  JsonDocument doc;
  doc["mac_address"] = DEVICE_MAC;
  doc["type"] = DEVICE_TYPE;
```

```
18    doc["software_version"] = get_current_firmware_version();
19    doc["connection_type"] = "wifi";
20
21    JsonObject telemetry = doc["telemetry"].to<JsonObject>();
22    telemetry["uptime_seconds"] = millis() / 1000;
23    telemetry["free_heap_bytes"] = ESP.getFreeHeap();
24    telemetry["wifi_rssi"] = WiFi.RSSI();
25
26    String requestBody;
27    serializeJson(doc, requestBody);
28
29    int code = http.POST(requestBody);
30    ...
31  }
```

Codice 5.5: Invio dell'heartbeat con telemetria (versione sintetizzata)

La risposta del server contiene l'intervallo del prossimo heartbeat (limitato a un valore compreso tra 10 e 300 secondi, con default di 30), eventuali comandi da eseguire, le impostazioni di visualizzazione (intervallo di rotazione delle schermate, minimo 5 secondi) e l'elenco degli eventi. In caso di errore, il tentativo viene ripetuto dopo 15 secondi. La scadenza dei timer è calcolata con un confronto su differenza a 32 bit (`scaduto()`), che rimane corretto anche all'overflow del contatore `millis()`.

5.4.2 Cache degli eventi e persistenza

Gli eventi ricevuti (al massimo dieci, ciascuno con identificativo, nome, link per il QR code, numero di prenotazioni e di partecipanti) vengono confrontati con quelli già in memoria: solo se differiscono la cache RAM viene aggiornata e il contenuto è scritto nella memoria NVS tramite la libreria `Preferences`, limitando così le scritture sulla flash. Le chiavi relative a posizioni non più utilizzate vengono rimosse, e al termine viene richiesta la ricostruzione dell'interfaccia (Codice 5.6).

```
1  static void salva_eventi(JsonArray events, const char *origine) {
2    std::vector<ApiEvent> nuovi;
3    for (JsonObject ev : events) {
4      if (nuovi.size() >= MAX_EVENTS) break;
5      ApiEvent c;
6      c.id = ev["id"] | 0L;
7      c.name = ev["name"] | "";
8      c.qr = ev["qr"] | "";
9      c.reservations = ev["reservations"] | 0;
10     c.participants = ev["participants"] | 0;
11     nuovi.push_back(c);
12   }
```

```
13
14    bool uguale = (nuovi.size() == events_cache.size());
15    for (size_t i = 0; uguale && i < nuovi.size(); ++i) {
16      uguale = eventi_uguali(nuovi[i], events_cache[i]);
17    }
18    if (uguale) return;
19
20    events_cache = nuovi;
21
22    Preferences prefs;
23    prefs.begin("api-cache", false);
24    for (size_t i = 0; i < MAX_EVENTS; ++i) {
25      char kId[16], kName[16], kQr[16], kPart[16], kPartic[16];
26      snprintf(kId, sizeof(kId), "ev_id_%u", (unsigned)i);
27      snprintf(kName, sizeof(kName), "ev_name_%u", (unsigned)i);
28      ...
29      if (i < events_cache.size()) {
30        prefs.putLong(kId, events_cache[i].id);
31        prefs.putString(kName, events_cache[i].name);
32        ...
33      } else {
34        if (prefs.isKey(kId)) prefs.remove(kId);
35        ...
36      }
37    }
38    prefs.end();
39    ui_richiedi_ricarica_tileview();
40  }
```

Codice 5.6: Aggiornamento e salvataggio persistente degli eventi (versione sintetizzata)

Alla lettura (`api_get_cached_events()`), se la cache in RAM è vuota — ad esempio dopo un riavvio — gli eventi vengono ricostruiti dalla NVS. Il dispositivo è quindi in grado di mostrare gli ultimi dati noti anche in assenza di connettività. Per questo motivo la funzione `ui_load_cached_data()` invocata in `setup()` non deve eseguire alcuna operazione: il caricamento avviene in modo differito alla prima costruzione delle schermate.

5.4.3 WebSocket e comandi remoti

Oltre al polling periodico, il dispositivo stabilisce una connessione WebSocket persistente, i cui parametri (host, porta, chiave applicativa e canali) sono comunicati dal CRM nella risposta all'heartbeat. Il protocollo utilizzato è compatibile con Pusher: dopo l'handshake `pusher:connection_established` il client si iscrive a due canali, uno specifico del dispositivo e uno dell'organizzazione. Per i canali privati, la sottoscrizione richiede un token ottenuto con una richiesta POST all'endpoint `/broadcasting/auth`. Il client risponde ai messaggi `pusher:ping`, effettua la riconnessione automatica ogni 5

secondi e utilizza un heartbeat applicativo di 15 secondi.
Attraverso questo canale, e analogamente tramite il campo `commands` della risposta all'heartbeat, il CRM può inviare comandi remoti gestiti da `esegui_comando_remoto()`:

- `restart`: riavvio del dispositivo;
- `reset`: cancellazione delle credenziali Wi-Fi, delle impostazioni e della cache eventi, seguita dal riavvio; il logo salvato in LittleFS non viene eliminato da questo comando;
- `shutdown`: spegnimento del display e del Wi-Fi e ingresso in deep sleep;
- `ota`: aggiornamento del firmware a partire dall'URL indicato nel comando (`firmware_url`).

L'aggiornamento OTA è eseguito con `httpUpdate` usando un client Wi-Fi non cifrato. La funzione tenta di acquisire il mutex LVGL con un timeout di cinque secondi; se non riesce, procede comunque con il download, quindi la sospensione del rendering non è garantita in ogni situazione. In caso di aggiornamento riuscito il dispositivo viene riavviato.
Gli stessi messaggi WebSocket possono inoltre contenere un blocco `settings` o un elenco `events`, che vengono elaborati con le stesse funzioni dell'heartbeat, consentendo l'aggiornamento immediato del display quando i dati cambiano sul CRM.

5.4.4 Logo dell'organizzazione

Il `NetworkTask` scarica il logo dell'organizzazione fuori dal mutex LVGL, con un tentativo ogni 30 secondi finché non riesce, in modo da non bloccare l'interfaccia. Il server può restituire direttamente un file PNG oppure una sua codifica esadecimale incapsulata in JSON. In entrambi i casi il contenuto è validato (firma PNG, dimensioni massime 1024×1024 pixel e 1 MB) e salvato in LittleFS nel file `/logo.png`, così da essere disponibile anche dopo un riavvio.

5.5 Frontend sul display: LVGL

L'interfaccia grafica si appoggia sui driver hardware del display AMOLED e del touch screen capacitivo configurati in `display_config.cpp`. La funzione `init_display_touch_lvgl()` avvia il display, inizializza il touch, alloca due buffer di rendering in PSRAM (la memoria viene verificata e, se non disponibile, il firmware si arresta) e configura LVGL 9 in modalità di rendering parziale a doppio buffer con formato colore RGB565. Un timer hardware incrementa il tick di LVGL ogni 2 ms, mentre un callback di arrotondamento

(`rounder_cb`) forza le aree invalidate ad avere coordinate iniziali pari e finali dispari, come richiesto dal controller CO5300 (Codice 5.7).

```
void init_display_touch_lvgl() {
  gfx->begin(80000000);
  ((Arduino_CO5300 *)gfx)->setBrightness(255);
  gfx->fillScreen(RGB565_BLACK);

  init_touch();
  lv_init();

  uint32_t buffer_pixels = LCD_WIDTH * (LCD_HEIGHT / 2);
  buf1 = (uint16_t *)heap_caps_aligned_alloc(64, buffer_pixels *
sizeof(uint16_t),
                                             MALLOC_CAP_SPIRAM |
MALLOC_CAP_8BIT);
  buf2 = (uint16_t *)heap_caps_aligned_alloc(64, buffer_pixels *
sizeof(uint16_t),
                                             MALLOC_CAP_SPIRAM |
MALLOC_CAP_8BIT);

  display = lv_display_create(screenWidth, screenHeight);
  lv_display_set_color_format(display, LV_COLOR_FORMAT_RGB565);
  lv_display_set_buffers(display, buf1, buf2, buffer_pixels *
sizeof(uint16_t),
                         LV_DISPLAY_RENDER_MODE_PARTIAL);

  lv_display_set_flush_cb(display, my_flush_cb);
  lv_display_add_event_cb(display, rounder_cb,
LV_EVENT_INVALIDATE_AREA, NULL);
  ...
}
```

Codice 5.7: Inizializzazione di display, touch e LVGL (versione sintetizzata)

Le due varianti hardware differiscono nella configurazione del touch e nei parametri del display. Nella variante da 1,75 pollici il touch è gestito dalla libreria `TouchDrvCSTXXX` con un controller della famiglia CST92xx; le coordinate sono adattate all'orientamento tramite le costanti `TOUCH_INVERT_X`, `TOUCH_INVERT_Y` e `TOUCH_SWAP_XY`. Nella variante da 1,8 pollici il controller CST820 è gestito tramite il driver della famiglia CST816x, collegato via I²C; il display usa il bus QSPI a 40 MHz e buffer alti un decimo dello schermo. L'avvio del pannello passa inoltre da un expander I/O; per il touch sono presenti sia un tentativo di disabilitare l'autosleep sia un debounce di 50 ms sul rilascio (Codice 5.8).

```
static void my_touchpad_read(lv_indev_t *indev, lv_indev_data_t
*data) {
  static int32_t lastX = 0, lastY = 0;
  static bool pressed = false;
```

```
  static uint32_t last_finger_ms = 0;

  int32_t fingers = CST820->IIC_Read_Device_Value(
      CST820-
>Arduino_IIC_Touch::Value_Information::TOUCH_FINGER_NUMBER);

  if (fingers > 0) {
    lastX = CST820->IIC_Read_Device_Value(
        CST820-
>Arduino_IIC_Touch::Value_Information::TOUCH_COORDINATE_X);
    lastY = CST820->IIC_Read_Device_Value(
        CST820-
>Arduino_IIC_Touch::Value_Information::TOUCH_COORDINATE_Y);
    pressed = true;
    last_finger_ms = millis();
  } else if (pressed && (millis() - last_finger_ms) >
TOUCH_RELEASE_DEBOUNCE_MS) {
    pressed = false;
  }

  CST820->IIC_Interrupt_Flag = false;
  data->state = pressed ? LV_INDEV_STATE_PRESSED :
LV_INDEV_STATE_RELEASED;
  data->point.x = lastX;
  data->point.y = lastY;
}
```

Codice 5.8: Lettura del touch con debounce nella variante con controller CST820

5.5.1 Struttura e navigazione delle schermate

La funzione `create_tileview()` costruisce dinamicamente l'insieme delle schermate a partire dalla cache degli eventi: una schermata per ciascun evento attivo, una schermata fissa con il logo dell'organizzazione e una schermata fissa di menu e diagnostica. Se non è disponibile alcun evento, viene mostrato un segnaposto («In attesa di eventi Cloud…»).
Ogni schermata evento contiene un `tileview` a due tile: la prima riporta identificativo, nome, numero di prenotazioni e di partecipanti, mentre la seconda, raggiungibile con uno scorrimento orizzontale, mostra il QR code generato dal modulo `lv_qrcode` di LVGL a partire dal link ricevuto dal CRM. La schermata del logo decodifica il file PNG con `lodepng`, converte i pixel da RGBA a RGB565 e li fornisce a LVGL tramite un descrittore d'immagine; se il logo non è disponibile compare un messaggio di cortesia.
La navigazione tra le schermate avviene in due modi. Una rotazione automatica, la cui cadenza è fornita dal CRM (30 secondi di default), avanza ciclicamente le schermate con un'animazione di transizione. In alternativa, due

pulsanti fissi, collocati su un livello sovrapposto (`lv_layer_top`), permettono di scorrere manualmente in avanti e indietro. Un triplo tocco entro 1,5 secondi attiva o disattiva la rotazione automatica, e lo stato viene segnalato da un alone luminoso sui pulsanti. Un flag di animazione in corso impedisce cambi di schermata concorrenti.
La schermata di menu consente di visualizzare lo stato della rete, l'SSID quando connesso e la versione del firmware, mostrare i QR code per accedere al portale di configurazione o condividere la rete corrente, ed eseguire operazioni locali di manutenzione. La voce visualizzata come "Ultima sincronizzazione" rappresenta in realtà il tempo trascorso dall'ultima disconnessione Wi-Fi, non l'istante dell'ultima sincronizzazione con il CRM.

5.6 Frontend web: configurazione e diagnostica

Il portale web di configurazione sfrutta file statici salvati su LittleFS e serviti dal server HTTP integrato. Le pagine eseguono chiamate asincrone per aggiornare periodicamente le etichette di diagnostica e inviare comandi di configurazione Wi-Fi senza richiedere una connessione a Internet esterna. Per consentire alle pagine di riallinearsi dopo un cambio di stato, l'endpoint `/status` restituisce anche un contatore (`state_epoch`) incrementato a ogni modifica della configurazione Wi-Fi, oltre ai secondi trascorsi dalla perdita di connessione.

5.7 Aspetti di sicurezza e limiti osservati

Dall'analisi del codice sono emerse le seguenti criticità, da affrontare prima del rilascio in produzione:

- **Segreti e token API**: il token di autenticazione è attualmente definito in chiaro all'interno del codice sorgente (`api.cpp`), così come l'indirizzo MAC identificativo del dispositivo; è necessario adottare un sistema di archiviazione sicuro o una configurazione esterna per ciascun dispositivo.
- **Validazione TLS**: l'utilizzo di `setInsecure()` sul client TLS disabilita il controllo dei certificati del server remoto; è raccomandata l'implementazione della validazione della catena di trust. Considerazioni analoghe valgono per la connessione WebSocket sicura.
- **Access Point di configurazione**: le credenziali dell'Access Point (`ESP32-Config`) sono fisse e comuni a tutti i dispositivi, e gli endpoint di gestione (`/reboot`, `/reset_factory`, `/connect`) non richiedono alcuna autenticazione; chiunque si colleghi all'Access Point può quindi modificarne la configurazione. Le credenziali della rete Wi-Fi sono inoltre memorizzate in chiaro nella NVS.

Aggiornamenti OTA: l'aggiornamento è attivabile tramite comando remoto; il controllo periodico autonomo è disattivato e la funzione dedicata restituisce sempre `false`. Il download usa un client non cifrato e nel codice esaminato non è presente una verifica esplicita della firma dell'immagine; prima di un impiego produttivo sono quindi necessari un trasporto autenticato e controlli d'integrità e autenticità. **Operazioni bloccanti**: la richiesta `/connect` attende fino a 10 secondi l'esito della connessione all'interno del `NetworkTask`, sospendendo nel frattempo l'elaborazione periodica di DNS, HTTP, heartbeat e WebSocket.

- **Reset con semantiche diverse**: il reset locale dal menu cancella le credenziali Wi-Fi e la cache eventi, ma non tutte le impostazioni né il logo; il comando remoto `reset` cancella anche il namespace delle impostazioni, ma non elimina il logo. L'endpoint locale `/reset_factory` è invece quello che rimuove anche il file del logo. Le diverse operazioni non hanno quindi lo stesso effetto e andrebbero uniformate.
- **Stato di sincronizzazione**: la finestra diagnostica presenta come ultima sincronizzazione il tempo dalla disconnessione della rete, mentre quando la rete è connessa mostra "ora"; non viene mantenuto un timestamp effettivo dell'ultimo scambio riuscito con il CRM.

6 Verifica e validazione

La fase di verifica ha avuto lo scopo di controllare che le singole funzioni del firmware rispondessero ai requisiti definiti durante l'analisi; la validazione ha invece riguardato l'uso del sistema completo nelle condizioni operative previste. Poiché il progetto integra componenti che dipendono direttamente da periferiche fisiche, entrambe le attività sono state svolte sulla macchina reale: le schede ESP32-S3 con display e touch sono state alimentate e programmate, e le funzioni sono state provate interagendo con il pannello, con il portale Wi-Fi e con il servizio RelAi. Non è stato utilizzato un simulatore per sostituire il display, il controller touch o la connettività del dispositivo.

6.1 Ambiente e modalità di prova

Il firmware è stato sviluppato e caricato sulle schede mediante PlatformIO, usando l'ambiente `esp32-s3-devkitc-1`. Durante le prove il comportamento del dispositivo è stato osservato direttamente sul display e, quando necessario, attraverso il monitor seriale configurato a 115200 baud. I messaggi prodotti dai moduli Wi-Fi, API, WebSocket, cache, LittleFS e interfaccia hanno permesso di seguire l'avvio, l'esito delle richieste al CRM e gli aggiornamenti della cache senza ricorrere a strumenti di simulazione.
Le prove hanno riguardato la configurazione principale con display rotondo da 1,75 pollici e la seconda variante da 1,8 pollici. Le verifiche relative ai servizi di rete e alla sincronizzazione sono state svolte sul prototipo connesso alla rete e all'ambiente RelAi disponibile; le prove di visualizzazione, navigazione e touch sono state effettuate sul pannello fisico e ripetute sulla variante hardware interessata. In questo modo è stato possibile verificare non soltanto la correttezza dei dati ricevuti, ma anche il loro effettivo rendering e la risposta dell'interfaccia ai tocchi.
La directory `test` del progetto non contiene una suite di test automatizzati, ma soltanto il file introduttivo predefinito di PlatformIO. Il collaudo è quindi di tipo funzionale e manuale: per ciascuna prova si è eseguita l'operazione sul dispositivo, osservando il risultato sul display, nel portale o nel monitor seriale. Gli esiti descritti di seguito sono qualitativi; non vengono attribuiti al sistema valori numerici di latenza, autonomia o copertura che non siano stati raccolti con misurazioni dedicate.

6.2 Avvio e configurazione della rete

La prima verifica è stata eseguita dopo il caricamento del firmware e il riavvio della scheda. Sul monitor seriale è stato controllato il completamento

dell'inizializzazione del file system LittleFS e del routing HTTP; sul dispositivo è stata verificata la comparsa dell'Access Point `ESP32-Config`. Questo passaggio ha permesso di accertare che il sistema fosse raggiungibile anche prima di aver configurato una rete esterna.
La procedura di provisioning è stata provata collegandosi all'Access Point con un dispositivo client e aprendo il portale locale. Dal portale è stata avviata la scansione delle reti, selezionata una rete disponibile e inviata la richiesta di connessione. In presenza di credenziali valide, il firmware ha completato la connessione station e salvato SSID e password nella NVS; dopo il riavvio è stato verificato che le credenziali fossero ancora disponibili e che la connessione venisse avviata nuovamente.
È stato provato anche il caso di credenziali errate. Il firmware interrompe il tentativo non riuscito, restituisce al portale un esito di errore e mantiene attivo l'Access Point, così da permettere una nuova configurazione senza riflashare la scheda. La prova è importante perché verifica insieme la gestione dell'errore e la raggiungibilità del dispositivo dopo un input non valido.
Successivamente sono stati consultati gli endpoint locali di stato e scansione. La risposta di `/status` è stata confrontata con lo stato effettivo della connessione e con la versione del firmware; `/scan` è stato verificato durante l'avvio asincrono e al completamento della ricerca. Le richieste di connessione e di reset sono state controllate dal portale, considerando che i diversi percorsi di reset non hanno tutti la stessa semantica, come illustrato nella sezione dedicata ai limiti.

6.3 Sincronizzazione con RelAi e funzionamento offline

Con il dispositivo connesso alla rete, è stata verificata la comunicazione periodica con il CRM tramite heartbeat HTTP. Il controllo ha riguardato l'invio delle informazioni identificative e della telemetria disponibili nel firmware, tra cui uptime, memoria heap libera e intensità del segnale Wi-Fi, insieme alla ricezione delle impostazioni e degli eventi. Gli eventi ricevuti sono stati confrontati con quelli visualizzati sul display: per ciascun elemento devono essere coerenti identificativo, nome, prenotazioni, partecipanti e collegamento associato al QR code.
È stata verificata anche la ricezione degli aggiornamenti attraverso il canale WebSocket. Dopo l'attivazione del canale da parte della configurazione ricevuta dal CRM, le variazioni di eventi e impostazioni vengono elaborate dal task di rete; se i dati sono cambiati, il firmware aggiorna la cache e richiede la ricostruzione delle schermate. Il risultato atteso, osservato durante la prova, è che le informazioni sul pannello si aggiornino senza riavviare il dispositivo né

intervenire manualmente sul display.
Per il funzionamento offline è stata interrotta la connessione al router e sono state poi ripetute le verifiche dopo un riavvio. Gli eventi precedentemente acquisiti vengono recuperati dalla NVS, mentre il logo salvato viene letto da LittleFS: la schermata può quindi continuare a mostrare l'ultimo contenuto disponibile anche quando il CRM non è raggiungibile. Quando la rete risulta assente per oltre quindici secondi e sono presenti credenziali salvate, il task di rete mostra inoltre un avviso sul display; al ripristino del collegamento l'avviso viene rimosso e la sincronizzazione può riprendere.
Il logo è stato verificato separatamente controllando il download quando la rete è connessa, la persistenza dopo il riavvio e la visualizzazione della schermata dedicata. Il firmware accetta il contenuto PNG binario oppure una codifica esadecimale compatibile, controlla la firma del file e i limiti dimensionali dichiarati nel codice, e memorizza il risultato in LittleFS. Quando il logo non è disponibile o non è decodificabile, l'interfaccia mostra un messaggio alternativo invece di lasciare una schermata incompleta.

6.4 Verifica dell'interfaccia sui dispositivi fisici

La verifica grafica è stata condotta osservando il display durante la creazione delle pagine evento, la visualizzazione del logo e l'apertura del menu. Per gli eventi sono stati controllati i valori ricevuti dal CRM e la corrispondenza tra ogni pagina e il relativo codice QR. È stata poi verificata la navigazione orizzontale tra la scheda informativa e quella contenente il QR, oltre alla rotazione automatica e ai comandi manuali di avanzamento e arretramento.
Le interazioni sono state eseguite direttamente sul touch screen, non simulate via software. Sono stati controllati i pulsanti di navigazione, le finestre modali di stato e di condivisione Wi-Fi e il triplo tocco che abilita o sospende la rotazione automatica. Sul modello con controller CST820 è stata inoltre verificata la risposta al rilascio del dito: il firmware applica un debounce di 50 ms e tenta di disabilitare la modalità di autosleep del controller, mentre la variante da 1,75 pollici usa il driver CST della famiglia CST92xx e le proprie trasformazioni delle coordinate.
Il confronto tra le due schede ha permesso di verificare che la gestione del display e del touch dipenda dalla configurazione hardware, mentre la logica di presentazione dei dati e di sincronizzazione rimanga comune. In particolare, sono stati considerati l'avvio del pannello, il rendering dei contenuti e la possibilità di raggiungere le funzioni principali attraverso i controlli previsti. Le verifiche non sono state limitate alla compilazione: l'esito è stato valutato sul dispositivo acceso e durante l'interazione fisica con lo schermo.

6.5 Anomalie emerse e interventi correttivi

Le prove sul prototipo sono state utili anche per individuare problemi che non sarebbero risultati evidenti dalla sola lettura dei sorgenti. Una prima anomalia riguardava il salvataggio del logo nella NVS: la dimensione dell'immagine poteva superare lo spazio disponibile per i dati gestiti da `Preferences`, causando errori durante il funzionamento e, in alcune condizioni, il riavvio del dispositivo. La persistenza del logo è stata quindi spostata su LittleFS, mentre la NVS è rimasta dedicata a credenziali, impostazioni e cache strutturata degli eventi.
Un secondo problema era la scarsa reattività dell'interfaccia durante il recupero del logo. La richiesta HTTP e la successiva elaborazione non devono occupare il mutex di LVGL: il download è stato collocato nel task di rete, al di fuori della sezione critica grafica, e la UI legge il file già disponibile. Questo intervento ha separato il lavoro di rete dal rendering e ridotto il rischio che una richiesta lenta impedisse l'aggiornamento dello schermo.
Durante le prove del touch sulla variante da 1,8 pollici è stata inoltre affrontata la perdita di alcuni tocchi dopo un periodo di inattività, riconducibile alla modalità di autosleep del controller. La configurazione prova a disabilitare tale modalità e la lettura conferma il rilascio solo dopo un intervallo di debounce. L'interazione è stata quindi verificata nuovamente sul pannello fisico dopo la correzione, prestando attenzione sia alla pressione sia al rilascio.

6.6 Esito della validazione e limiti residui

Il collaudo sulla macchina fisica ha consentito di validare i flussi principali previsti dai requisiti: configurazione Wi-Fi, ricezione e visualizzazione degli eventi, aggiornamento dei contenuti, recupero dei dati locali dopo una disconnessione e navigazione touch. La combinazione tra osservazione del display e riscontri seriali ha permesso di seguire il comportamento dalla richiesta di rete fino alla presentazione dei dati all'utente. Gli esiti sono qualitativi e riferiti alle prove funzionali svolte sul prototipo, non a una campagna statistica su un campione di dispositivi.
Rimangono alcuni limiti da considerare. Non sono presenti test automatici né registrazioni strutturate con tempi di risposta o misure di stabilità a lungo termine. La funzione di stampa periodica delle statistiche heap è presente nel codice, ma la relativa chiamata risulta commentata nel task di rete: non è quindi possibile ricavare dal firmware attuale una serie temporale affidabile del consumo di memoria. Le prove di durata svolte durante lo stage hanno permesso di osservare il sistema in uso prolungato, ma non sono accompagnate

da dati sufficienti per dichiarare una soglia quantitativa di affidabilità.
Anche il percorso di aggiornamento OTA richiede ulteriori verifiche prima di un impiego produttivo: il controllo periodico autonomo non è attivo, il trasferimento usa un client non cifrato e nel codice esaminato non è presente una verifica esplicita della firma del firmware. Infine, le procedure chiamate reset hanno effetti differenti: il reset dal menu elimina le credenziali Wi-Fi e la cache eventi, il comando remoto cancella anche il namespace delle impostazioni ma non il logo, mentre l'endpoint `/reset_factory` rimuove anche il file del logo. Questa differenza è stata riportata come limite funzionale e rappresenta un possibile intervento futuro per rendere uniforme il ripristino di fabbrica.
Nel complesso, le prove eseguite direttamente sulle schede hanno confermato la coerenza delle funzionalità essenziali con i casi d'uso descritti. Per rendere il processo ripetibile e ampliare la copertura, un'evoluzione del progetto dovrebbe affiancare al collaudo hardware test automatici per parsing e cache, prove d'integrazione documentate per gli endpoint e il CRM e la raccolta sistematica dei log e delle misure durante le sessioni prolungate.

7 Conclusioni

Il progetto di stage ha affrontato la progettazione e la realizzazione di un'interfaccia embedded capace di presentare su display le informazioni gestite dal CRM RelAi. Il lavoro ha integrato aspetti diversi: configurazione della rete, comunicazione con servizi remoti, persistenza locale dei dati, gestione di periferiche grafiche e progettazione dell'interazione touch. Le attività si sono svolte in modo iterativo, alternando implementazione, prove sulla macchina e confronto con il tutor aziendale.

7.1 Consuntivo del progetto

Il risultato dello stage è un prototipo basato su ESP32-S3, sviluppato per due configurazioni hardware con display da 1,75 e 1,8 pollici. Il firmware avvia un Access Point utilizzabile per la configurazione Wi-Fi, si collega alla rete dell'organizzazione quando sono disponibili credenziali valide e comunica con RelAi tramite heartbeat HTTP e WebSocket. I dati degli eventi ricevuti vengono organizzati in una cache locale persistente, così da poter essere consultati anche in seguito a riavvio o in assenza temporanea della rete.
Sul display vengono presentate schermate dedicate agli eventi, ai rispettivi codici QR, al logo dell'organizzazione e alle funzioni di menu e diagnostica. La navigazione può avvenire attraverso i controlli touch o mediante rotazione automatica. La separazione tra task di rete e task grafico, insieme alla sincronizzazione dell'accesso a LVGL, consente di organizzare le operazioni concorrenti del sistema. Nel corso dello sviluppo sono state affrontate criticità specifiche dell'hardware, tra cui la persistenza del logo, la reattività dell'interfaccia durante le operazioni di rete e la risposta del touch sulla variante da 1,8 pollici.
Il prototipo è stato compilato e provato sulle schede fisiche. Il collaudo ha compreso l'interazione diretta con display e touch, le procedure di provisioning, l'osservazione degli aggiornamenti ricevuti e il recupero dei dati locali. La verifica è stata prevalentemente manuale e qualitativa: il repository non contiene test automatici e non sono disponibili misurazioni sistematiche delle prestazioni o della stabilità a lungo termine. Il risultato va pertanto considerato una base funzionante verificata sui dispositivi disponibili, non una certificazione di prodotto per il rilascio in produzione.

7.2 Raggiungimento degli obiettivi

Gli obiettivi fondamentali dello stage, relativi alla visualizzazione delle informazioni su hardware embedded e all'integrazione con RelAi, sono stati

concretizzati nel prototipo. Sono presenti la configurazione Wi-Fi locale, la ricezione e la visualizzazione degli eventi, la generazione dei codici QR, la navigazione tra schermate e il recupero della cache dopo una disconnessione o un riavvio. La validazione svolta sulla macchina ha permesso di controllare questi flussi nel contesto d'uso reale, includendo le differenze tra le due varianti di display.
Alcuni obiettivi devono essere interpretati con maggiore cautela. La comunicazione near-real-time è supportata dal canale WebSocket, ma la configurazione remota della rotazione non aggiorna necessariamente il timer già attivo, poiché il valore ricevuto viene conservato nel modulo API e letto alla creazione del timer. La diagnostica è disponibile sul display e tramite endpoint del portale locale, ma non tutte le informazioni rappresentano una reale ultima sincronizzazione con il CRM. L'aggiornamento OTA è predisposto e attivabile da comando remoto, ma non è attivo un controllo periodico autonomo e il percorso necessita di ulteriori misure di sicurezza. Inoltre, le diverse procedure di reset non cancellano esattamente lo stesso insieme di dati. Questi aspetti non annullano il raggiungimento degli obiettivi principali, ma definiscono con precisione il perimetro del prototipo e le funzionalità da consolidare.

7.3 Competenze acquisite

L'attività ha permesso di approfondire lo sviluppo firmware in C++ su ESP32-S3 e l'organizzazione di applicazioni basate su FreeRTOS. In particolare, sono state acquisite competenze nella gestione di task concorrenti, mutex e risorse condivise, oltre che nell'integrazione di LVGL con display e controller touch differenti.
La parte di rete ha richiesto di lavorare con Wi-Fi in modalità AP e STA, server HTTP locale, DNS per il captive portal, richieste HTTP autenticate e connessioni WebSocket. È stata inoltre approfondita la gestione della memoria non volatile tramite NVS, del file system LittleFS e delle strategie di cache per mantenere disponibili i dati durante le interruzioni di rete. Il lavoro ha evidenziato l'importanza di verificare le ipotesi direttamente sul dispositivo: tempi di risposta, comportamento del touch e disponibilità di memoria dipendono dall'interazione tra firmware, librerie e componenti fisici.
Dal punto di vista del processo, lo sviluppo iterativo, le prove frequenti e il confronto con il tutor hanno consentito di rivedere le scelte tecniche sulla base dei problemi osservati. La documentazione dei moduli e la ricostruzione dei flussi di errore hanno contribuito a sviluppare un approccio più sistematico alla diagnosi e alla manutenzione del software embedded.

7.4 Limiti e sviluppi futuri

Prima di un eventuale impiego produttivo, il firmware dovrebbe essere sottoposto a un ciclo di verifica più ripetibile e documentato. Un primo intervento consiste nell'introdurre test automatici per il parsing dei payload, il confronto e la persistenza della cache, affiancati da prove d'integrazione degli endpoint e del flusso di aggiornamento con un ambiente CRM dedicato. Per le prove sulla macchina sarebbe utile conservare i log e annotare configurazione hardware, versione firmware, condizioni di rete ed esito, così da rendere confrontabili le sessioni.
Sul piano funzionale è opportuno uniformare le operazioni di reset, rendere effettivo l'aggiornamento delle impostazioni remote durante l'esecuzione e registrare un timestamp reale dell'ultima sincronizzazione riuscita. Le sessioni di durata dovrebbero inoltre monitorare in modo sistematico heap libero, blocco massimo disponibile, riavvii e perdita di connettività, per poter valutare la stabilità con dati osservabili.
La sicurezza richiede particolare attenzione: il codice esaminato contiene credenziali di autenticazione definite nel sorgente, disabilita la verifica dei certificati TLS per le richieste HTTPS e utilizza un client non cifrato per il download OTA. Un'evoluzione dovrebbe adottare credenziali individuali gestite fuori dal codice condiviso, autenticazione verificata del server e controllo d'integrità e autenticità del firmware prima dell'installazione. Anche l'Access Point di configurazione e gli endpoint locali dovrebbero essere protetti in modo adeguato al contesto di distribuzione.
Infine, il supporto alle due varianti hardware offre una base per ampliare la compatibilità con ulteriori display, mentre la personalizzazione dei pannelli, dei temi e delle informazioni visualizzate può essere sviluppata in accordo con le necessità delle organizzazioni clienti. Tali estensioni dovrebbero seguire una priorità chiara: consolidare prima sicurezza e affidabilità, quindi ampliare le funzionalità.

7.5 Valutazione conclusiva

Il lavoro ha mostrato che è possibile portare informazioni provenienti da un CRM su un dispositivo compatto, configurabile localmente e utilizzabile senza interazione continua con un computer. La realizzazione ha richiesto di conciliare vincoli di memoria, aggiornamento grafico, connettività e differenze hardware, rendendo il collaudo fisico una parte essenziale del processo e non una semplice verifica finale.
Considero il risultato raggiunto un prototipo significativo rispetto agli obiettivi

dello stage: le funzioni centrali sono state implementate e provate sulle schede disponibili, mentre i limiti emersi sono stati esplicitati e forniscono indicazioni concrete per il lavoro successivo. L'esperienza ha consolidato le competenze nello sviluppo embedded e ha mostrato quanto la qualità di una soluzione dipenda non solo dalla sua funzionalità nominale, ma anche dalla capacità di osservarne il comportamento reale, riconoscere i vincoli e documentare con precisione ciò che è stato verificato.

7.6 Consuntivo finale

Esempio di aggiunta di un termine con glossario e acronimo:

Lorem *SDK* (Software Development Kit) ipsum dolor.

Nel successivo utilizzo apparira solo l'acronimo:

Lorem *SDK*.

Nel caso si voglia invece mettere solo il termine per esteso:

Lorem *Software Development Kit*.

7.7 Raggiungimento degli obiettivi

Esempio di termine con solo acronimo: Lorem *TSA*, ipsum dolor sit amet.

Termine costruito senza acronimo: *Nome del termine*, ipsum dolor sit amet.

7.8 Conoscenze acquisite

Lorem ipsum dolor, Lorem *API*.

Lorem ipsum dolor, Lorem *Application Program Interface*.

Si puo consultare la sezione Glossario per alcuni esempi di utilizzo.

7.9 Valutazione personale

7.10 Valutazione personale

8 Glossario

Il presente glossario raccoglie gli acronimi, i termini tecnici e i nomi di strumenti e tecnologie utilizzati nel documento, in ordine alfabetico. Le definizioni sono riferite al contesto del progetto: dove un termine ha più significati, è riportato quello adottato nel lavoro di stage.

Access Point (AP) Modalità di funzionamento Wi-Fi in cui il dispositivo crea una propria rete alla quale altri apparecchi possono collegarsi. Nel progetto la rete `ESP32-Config` permette di raggiungere il portale di configurazione. Si contrappone alla modalità *station*.

Agile Approccio alla gestione e allo sviluppo del software basato su iterazioni brevi, rilascio incrementale delle funzionalità e confronto continuo con il committente, che consente di adattarsi ai cambiamenti dei requisiti.

AMOLED *Active-Matrix Organic Light-Emitting Diode*. Tecnologia di display in cui ogni pixel emette luce propria, senza retroilluminazione, con neri profondi e consumi contenuti. È la tecnologia dei due display utilizzati nel prototipo.

API *Application Programming Interface*. Insieme di funzioni e protocolli che un sistema mette a disposizione per essere utilizzato da altri programmi. Nel progetto indica le interfacce HTTP del CRM RelAi con cui il dispositivo scambia dati.

Autosleep Funzione dei controller touch che riduce i consumi sospendendo la rilevazione dopo un periodo di inattività. Può causare la perdita dei primi tocchi, motivo per cui nella seconda variante hardware viene gestita con il *debounce* della lettura.

Bearer token Token di autenticazione inviato nell'header `Authorization` di una richiesta HTTP: chi lo possiede viene riconosciuto dal server come autorizzato. È utilizzato dal dispositivo per identificarsi presso il CRM RelAi.

Branch Ramo indipendente di un repository di controllo di versione, utilizzato per sviluppare una funzionalità o una correzione senza alterare il ramo principale.

Buffer grafico Area di memoria in cui la libreria grafica compone i pixel di una porzione di schermo prima di inviarli al display. Nel progetto sono due buffer allocati nella memoria PSRAM, usati in modalità di rendering parziale.

Cache locale Copia, conservata nel dispositivo, degli ultimi dati validi ricevuti dal CRM (elenco degli eventi). Permette di continuare a mostrare le informazioni in assenza di connettività e dopo un riavvio.

Captive portal Pagina web a cui viene automaticamente reindirizzato chi si collega a una rete Wi-Fi, prima di accedere a qualsiasi altra risorsa. Nel progetto serve a guidare l'utente nella configurazione della rete dell'organizzazione.

CO5300 Controller del display AMOLED pilotato dal microcontrollore tramite bus QSPI. Richiede che le aree da ridisegnare abbiano coordinate di inizio pari e di fine dispari.

Code review Revisione del codice sorgente da parte di un altro sviluppatore, finalizzata a controllarne qualità, leggibilità, rispetto delle convenzioni e correttezza prima dell'integrazione.

Controller touch CST Famiglia di controller per schermi touch capacitivi collegati via bus I^2C. Nelle due varianti hardware del progetto sono utilizzati i modelli CST92xx e CST820.

CRM *Customer Relationship Management*. Sistema software per la gestione delle relazioni con clienti e organizzazioni. Nel progetto è rappresentato da RelAi, con cui il dispositivo comunica.

Debounce Tecnica che filtra le variazioni spurie di un segnale di ingresso, considerando valido un cambiamento solo se persiste per un tempo minimo. Nel progetto viene usata per confermare il rilascio del tocco sul display.

Deep sleep Modalità a consumo minimo dell'ESP32 in cui quasi tutte le periferiche sono spente. Nel progetto viene raggiunta dal comando remoto `shutdown`.

DNS *Domain Name System*. Sistema che associa i nomi di dominio agli indirizzi IP. Il server DNS locale del dispositivo risponde a ogni richiesta con l'indirizzo dell'Access Point, così da reindirizzare i client al captive portal.

Endpoint Indirizzo (URL) a cui un servizio HTTP espone una determinata funzione, ad esempio `/scan` o `/status` sul server locale del dispositivo.

ESP32-S3 Microcontrollore prodotto da Espressif Systems con due core, connettività Wi-Fi e Bluetooth Low Energy e supporto alla memoria PSRAM esterna. È la piattaforma hardware su cui è eseguito il firmware.

Fallback Comportamento alternativo adottato quando la modalità normale non è disponibile. Nel progetto, in assenza di rete o del CRM, il dispositivo mostra i dati della cache locale e tenta la riconnessione in background.

Firmware Software eseguito direttamente da un microcontrollore e memorizzato nella sua memoria flash. Nel progetto indica l'intero programma del dispositivo, sviluppato in C++.

Flashing Operazione di caricamento del firmware nella memoria flash di un microcontrollore.

FreeRTOS Sistema operativo real-time per microcontrollori, integrato nell'ambiente di sviluppo dell'ESP32. Fornisce task, semafori, mutex e temporizzazioni; nel progetto permette di eseguire rete e grafica in parallelo sui due core.

Gitea Piattaforma open-source per l'hosting di repository Git, con gestione di branch, pull request e revisioni del codice.

Heartbeat Messaggio inviato a intervalli regolari da un dispositivo a un server per segnalare che è attivo. Nel progetto è una richiesta HTTP POST che trasmette anche la telemetria e riceve in risposta comandi, impostazioni ed eventi.

HTTP / HTTPS *HyperText Transfer Protocol* (*Secure*). Protocollo di comunicazione richiesta-risposta su cui si basa il web; la variante HTTPS cifra lo scambio mediante TLS.

I²C *Inter-Integrated Circuit.* Bus seriale a due fili (dati e clock) per la comunicazione tra un microcontrollore e le periferiche; nel progetto collega il controller touch.

Issue Segnalazione tracciata in uno strumento di gestione dei progetti, che descrive un requisito, un'attività o un difetto da risolvere.

JSON *JavaScript Object Notation.* Formato testuale leggero per lo scambio di dati strutturati, basato su coppie chiave-valore e liste. È il formato dei messaggi tra dispositivo e CRM.

LittleFS File system progettato per memorie flash di piccole dimensioni, resistente agli spegnimenti improvvisi. Nel progetto ospita le pagine del portale web e il file del logo dell'organizzazione.

lodepng Libreria C++ per la codifica e la decodifica di immagini PNG, utilizzata per convertire il logo ricevuto dal CRM in pixel visualizzabili su display.

LVGL *Light and Versatile Graphics Library.* Libreria grafica open-source per sistemi embedded, che fornisce componenti visuali, gestione degli eventi di input e rendering ottimizzato.

Message box Finestra modale che si sovrappone all'interfaccia per mostrare un'informazione o richiedere un'azione, bloccando l'interazione con il resto della schermata finché non viene chiusa.

MQTT *Message Queuing Telemetry Transport.* Protocollo leggero di tipo publish/subscribe per lo scambio di messaggi tra dispositivi IoT. È citato tra le alternative per l'aggiornamento near-real-time.

Mutex *Mutual exclusion.* Meccanismo di sincronizzazione che consente a un solo task alla volta di accedere a una risorsa condivisa. Nel progetto `lvgl_mutex` protegge gli oggetti grafici.

Near-real-time Aggiornamento dei dati con una latenza di pochi secondi dal momento in cui cambiano sul sistema di origine.

NVS *Non-Volatile Storage*. Area di memoria flash dell'ESP32 per salvare coppie chiave-valore che sopravvivono ai riavvii. È utilizzata tramite la libreria `Preferences` per credenziali Wi-Fi, impostazioni e cache degli eventi.

OTA *Over-the-Air*. Aggiornamento del firmware eseguito da remoto tramite la rete, senza collegare fisicamente il dispositivo a un computer.

Payload Contenuto utile di un messaggio o di una richiesta di rete, distinto dalle informazioni di intestazione; nel progetto è tipicamente un oggetto JSON.

Plane Piattaforma open-source per la gestione di progetti e il tracciamento di attività e issue, usata per monitorare l'avanzamento dello stage.

PlatformIO Ambiente di sviluppo per sistemi embedded, integrato con Visual Studio Code, che gestisce piattaforma di destinazione, librerie, compilazione e caricamento del firmware.

PNG *Portable Network Graphics*. Formato di immagine raster con compressione senza perdita e supporto alla trasparenza; è il formato del logo dell'organizzazione.

Provisioning Procedura di configurazione iniziale di un dispositivo. Nel progetto consiste nel collegarsi all'Access Point, scegliere la rete Wi-Fi e inserirne la password tramite il portale.

PSRAM *Pseudo-Static RAM*. Memoria RAM esterna al microcontrollore, più capiente della RAM interna, utilizzata per i buffer grafici di LVGL.

Pull request Richiesta di integrare le modifiche di un branch in un altro, che avvia la discussione e la revisione del codice.

Pusher (protocollo) Protocollo di messaggistica in tempo reale basato su WebSocket, con canali pubblici e privati a cui i client si iscrivono. Il CRM RelAi ne espone uno compatibile, usato per inviare comandi e aggiornamenti al dispositivo.

QR code *Quick Response code*. Codice a barre bidimensionale che codifica un testo, ad esempio un indirizzo web o le credenziali di una rete Wi-Fi, leggibile con la fotocamera di uno smartphone.

QSPI *Quad Serial Peripheral Interface*. Variante del bus SPI con quattro linee dati, che aumenta la velocità di trasferimento; nel progetto collega il display AMOLED al microcontrollore.

RelAi Piattaforma CRM di Spazio Dev S.r.l. con cui il dispositivo si integra: fornisce eventi e impostazioni e riceve la telemetria.

Rendering Processo con cui l'interfaccia viene disegnata in pixel. Nel progetto LVGL opera in modalità parziale: ridisegna solo le aree modificate, a blocchi, in due buffer alternati.

Reset di fabbrica Operazione che cancella configurazioni, credenziali e cache salvate e riporta il dispositivo allo stato iniziale, con l'Access Point di provisioning attivo.

REST *Representational State Transfer*. Stile architetturale per servizi web in cui le risorse sono identificate da URL e manipolate con i metodi HTTP (GET, POST ecc.).

RGB565 Formato colore a 16 bit per pixel con 5 bit per il rosso, 6 per il verde e 5 per il blu, usato per ridurre memoria e banda verso il display.

RSSI *Received Signal Strength Indicator*. Misura della potenza del segnale Wi-Fi ricevuto, espressa in dBm; è inclusa nella telemetria inviata al CRM.

Station (STA) Modalità Wi-Fi in cui il dispositivo si collega a una rete esistente, come un normale client. Nel progetto convive con l'Access Point (modalità AP+STA).

Task Unità di esecuzione di FreeRTOS, con proprio stack e priorità. Il firmware usa un `NetworkTask` sul core 0 e un `DisplayTask` sul core 1.

Telemetria Insieme di dati di stato misurati dal dispositivo e inviati al sistema centrale. Nel progetto comprende uptime, memoria heap libera e RSSI.

Thread-safe Caratteristica di un codice o di una libreria che può essere usato in modo sicuro da più task contemporaneamente. LVGL non lo è, perciò l'accesso è serializzato con un mutex.

Tile / Tileview Componente LVGL che organizza il contenuto in riquadri affiancati, tra cui si scorre con un gesto. Nel progetto ogni evento è un tileview con una tile informativa e una con il QR code.

TLS *Transport Layer Security*. Protocollo crittografico che garantisce riservatezza e autenticità delle comunicazioni; usato da HTTPS e dai WebSocket sicuri. La verifica dei certificati è attualmente disattivata (`setInsecure()`).

Toolchain Insieme di strumenti (compilatore, linker, programmi di caricamento) necessario a produrre ed eseguire il firmware sulla piattaforma di destinazione.

Touch screen capacitivo Schermo sensibile al tocco che rileva la variazione di capacità elettrica causata dal dito.

Typst Sistema di composizione tipografica basato su markup, utilizzato per redigere la tesi.

UML *Unified Modeling Language*. Linguaggio di modellazione grafico per descrivere struttura e comportamento di un sistema software. I diagrammi dei casi d'uso ne sono un tipo.

Uptime Tempo trascorso dall'ultimo avvio del dispositivo, o dall'ultima disconnessione quando riferito alla rete.

Watchdog Temporizzatore hardware o software che riavvia il sistema se il programma smette di rispondere entro un tempo limite; protegge da blocchi in esercizio prolungato.

WebSocket Protocollo che mantiene aperto un canale bidirezionale persistente tra client e server, permettendo a entrambi di inviare messaggi in qualsiasi momento. Nel progetto serve a ricevere comandi e aggiornamenti dal CRM.

Wi-Fi Famiglia di tecnologie di rete locale senza fili basate sullo standard IEEE 802.11.

Riferimenti bibliografici e sitografici

Per la descrizione delle tecnologie, delle interfacce di programmazione e delle librerie impiegate nello sviluppo del firmware (inclusi driver hardware, gestione display AMOLED, touch, rete Wi-Fi, WebSocket, OTA e parsing JSON) sono state consultate le documentazioni ufficiali e i repository dei rispettivi progetti. I collegamenti seguenti rimandano alle pagine di riferimento utilizzate.

Documentazione tecnica e piattaforme

1. Espressif Systems, *ESP32-S3: Datasheet*, documentazione tecnica del microcontrollore. https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf. Consultato il 9 novembre 2026.
2. Espressif Systems, *Arduino ESP32 Core Documentation*, documentazione generale delle API e librerie di sistema per ESP32 (Wi-Fi Access Point, Station e scansione, WebServer, DNSServer, LittleFS e Preferences). https://docs.espressif.com/projects/arduino-esp32/en/latest/. Consultato il 9 novembre 2026.
3. Espressif Systems, *ESP-IDF Programming Guide — FreeRTOS*, documentazione dei task FreeRTOS e dei meccanismi di sincronizzazione (mutex) per ESP32-S3. https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-reference/system/freertos.html. Consultato il 9 novembre 2026.
4. LVGL, *LVGL Documentation*, documentazione ufficiale della libreria grafica embedded (v9), inclusi widget, tileview, QR code e gestione del display. https://lvgl.io/docs/open. Consultato il 9 novembre 2026.
5. PlatformIO, *PlatformIO Documentation*, documentazione dell'ambiente di sviluppo, della piattaforma ESP32 e del file di configurazione `platformio.ini`. https://docs.platformio.org/en/latest/. Consultato il 9 novembre 2026.

Librerie e driver hardware

1. MoonOnOurNation, *Arduino_GFX*, libreria grafica per display QSPI e driver CO5300 utilizzata per il rendering a 16-bit (`display_config.cpp`). https://github.com/moononournation/Arduino_GFX. Consultato il 9 novembre 2026.

2. Lewis He, *TouchDrv*, libreria di gestione per il chip touch CST92XX (`display_config.cpp`). https://github.com/lewisxhe/TouchDrv. Consultato il 9 novembre 2026.
3. Lode Vandevenne, *LodePNG*, libreria per la decodifica e il parsing di immagini PNG in formato raw per il logo aziendale (`interfaccia_ui.cpp`). https://github.com/lvandeve/lodepng. Consultato il 9 novembre 2026.
4. Benoît Blanchon, *ArduinoJson*, documentazione della libreria per la serializzazione e il parsing JSON in sistemi embedded (`api.cpp`, `wifi_conessione.cpp`). https://arduinojson.org/. Consultato il 9 novembre 2026.
5. Links2004, *arduinoWebSockets*, repository e documentazione della libreria WebSocket Client utilizzata per la comunicazione in tempo reale con il server (`api.cpp`). https://github.com/Links2004/arduinoWebSockets. Consultato il 9 novembre 2026.

Strumenti di supporto

1. draw.io, *diagrams.net Documentation*, documentazione dell'editor utilizzato per la stesura dei diagrammi UML e di flusso. https://www.drawio.com/docs/. Consultato il 9 novembre 2026.
2. Typst, *Typst Documentation*, riferimento per l'impaginazione, la formattazione e la gestione dei listati nel documento. https://www.typst.app/docs/. Consultato il 9 novembre 2026.